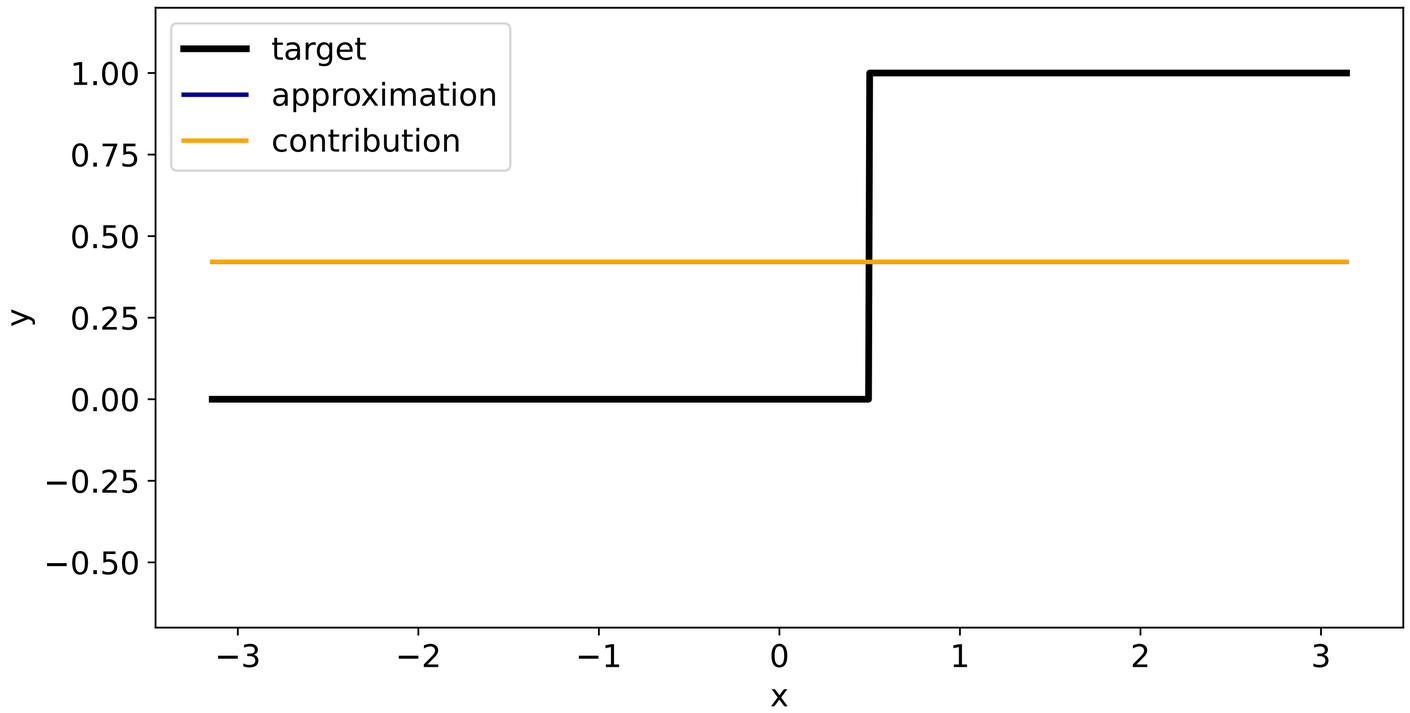
Attention and transformers

10-701 Introduction to Machine Learning

Geoff Gordon

Representing functions: Fourier basis

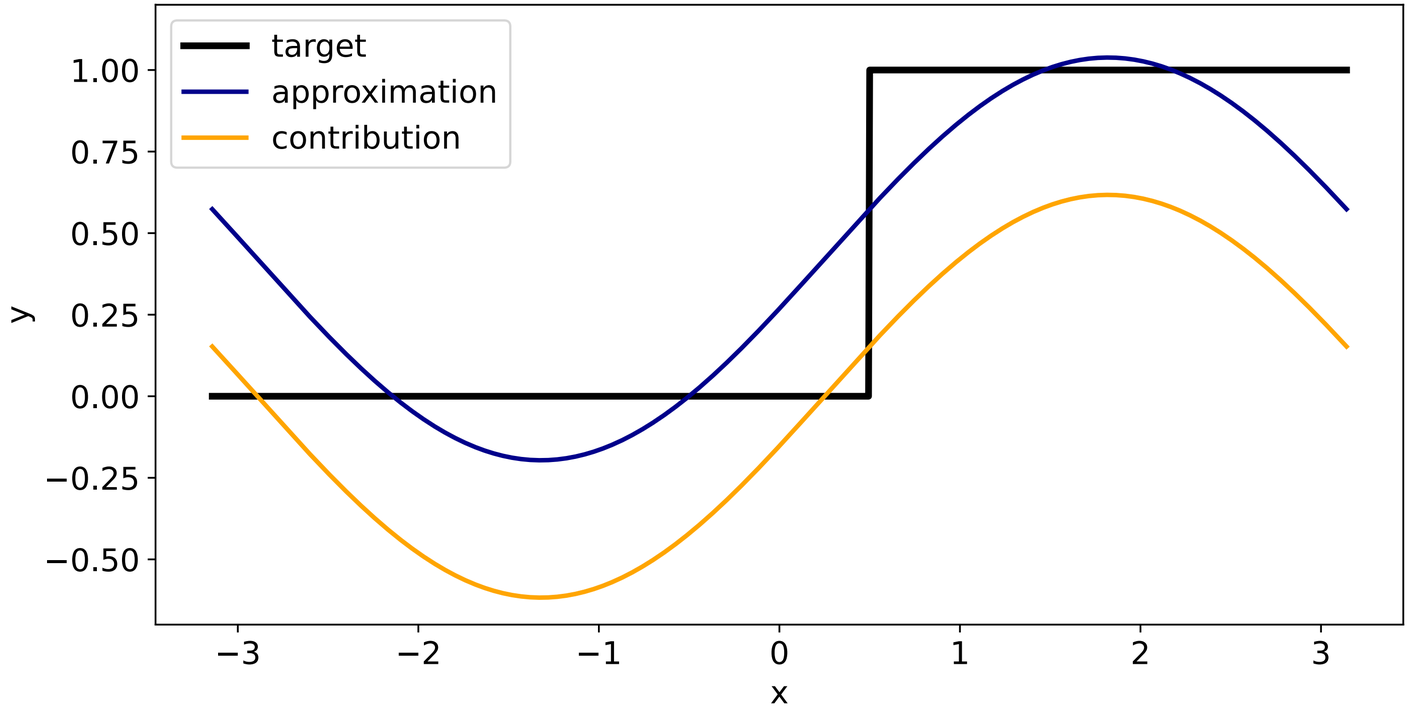


- One good way to represent functions is a Fourier basis:

$$f(\Delta) \approx \sum_{b=1}^{d} \alpha_b \sin(\theta_b \Delta) + \beta_b \cos(\theta_b \Delta) \qquad \Delta = j - i$$

  - with d different (fixed) frequencies θ_b, weights α_b, β_b

Representing functions: Fourier basis

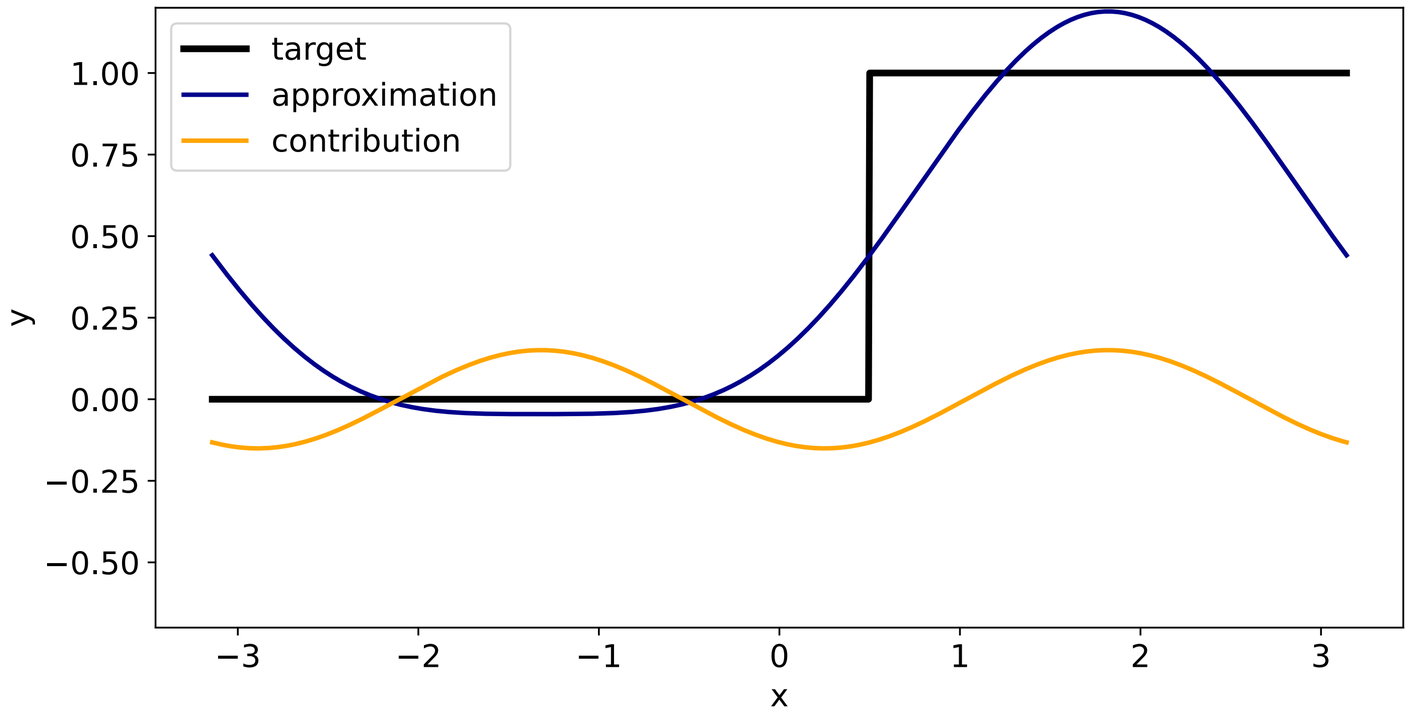


- One good way to represent functions is a Fourier basis:

$$f(\Delta) \approx \sum_{b=1}^{d} \alpha_b \sin(\theta_b \Delta) + \beta_b \cos(\theta_b \Delta) \qquad \Delta = j - i$$

  - with d different (fixed) frequencies θ_b, weights α_b, β_b

Representing functions: Fourier basis

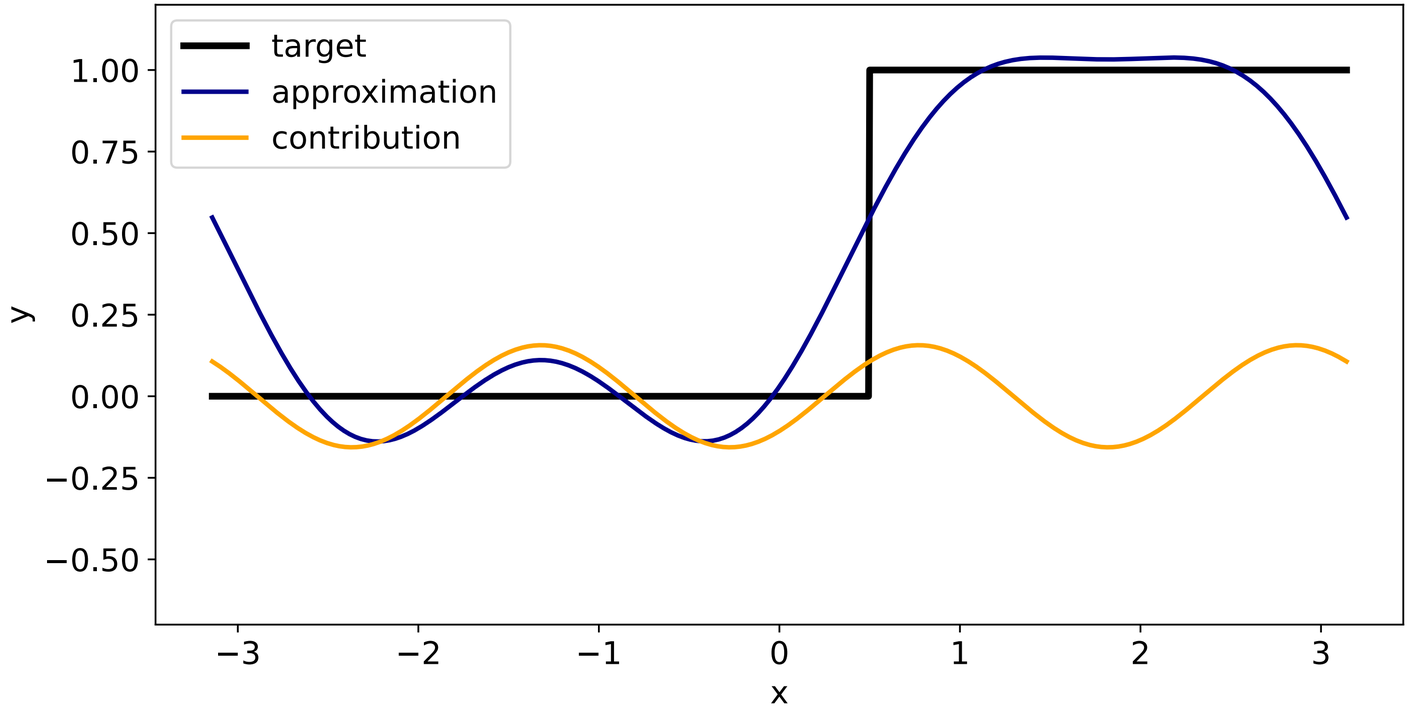


- One good way to represent functions is a Fourier basis:

$$f(\Delta) \approx \sum_{b=1}^{d} \alpha_b \sin(\theta_b \Delta) + \beta_b \cos(\theta_b \Delta) \qquad \Delta = j - i$$

  - with d different (fixed) frequencies θ_b, weights α_b, β_b

Representing functions: Fourier basis

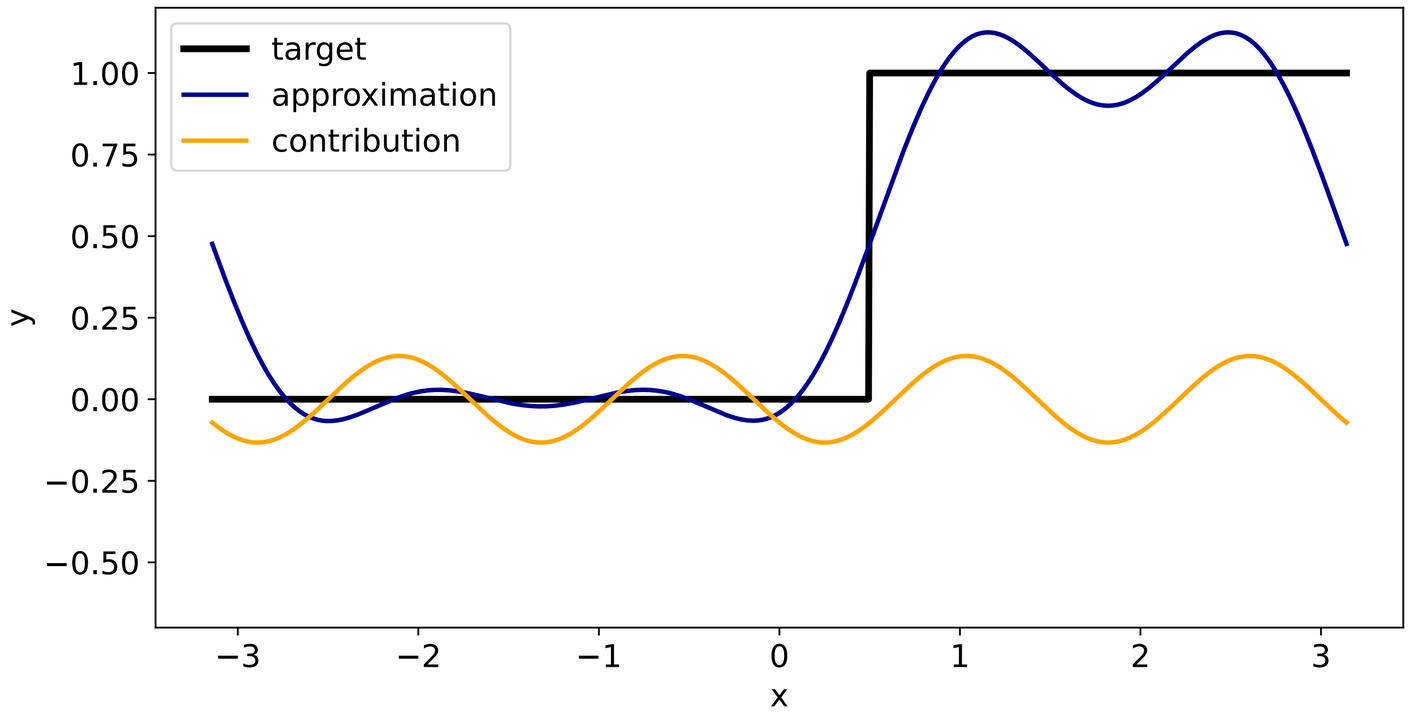


- One good way to represent functions is a Fourier basis:

$$f(\Delta) \approx \sum_{b=1}^{d} \alpha_b \sin(\theta_b \Delta) + \beta_b \cos(\theta_b \Delta) \qquad \Delta = j - i$$

  - with d different (fixed) frequencies θ_b, weights α_b, β_b

Representing functions: Fourier basis

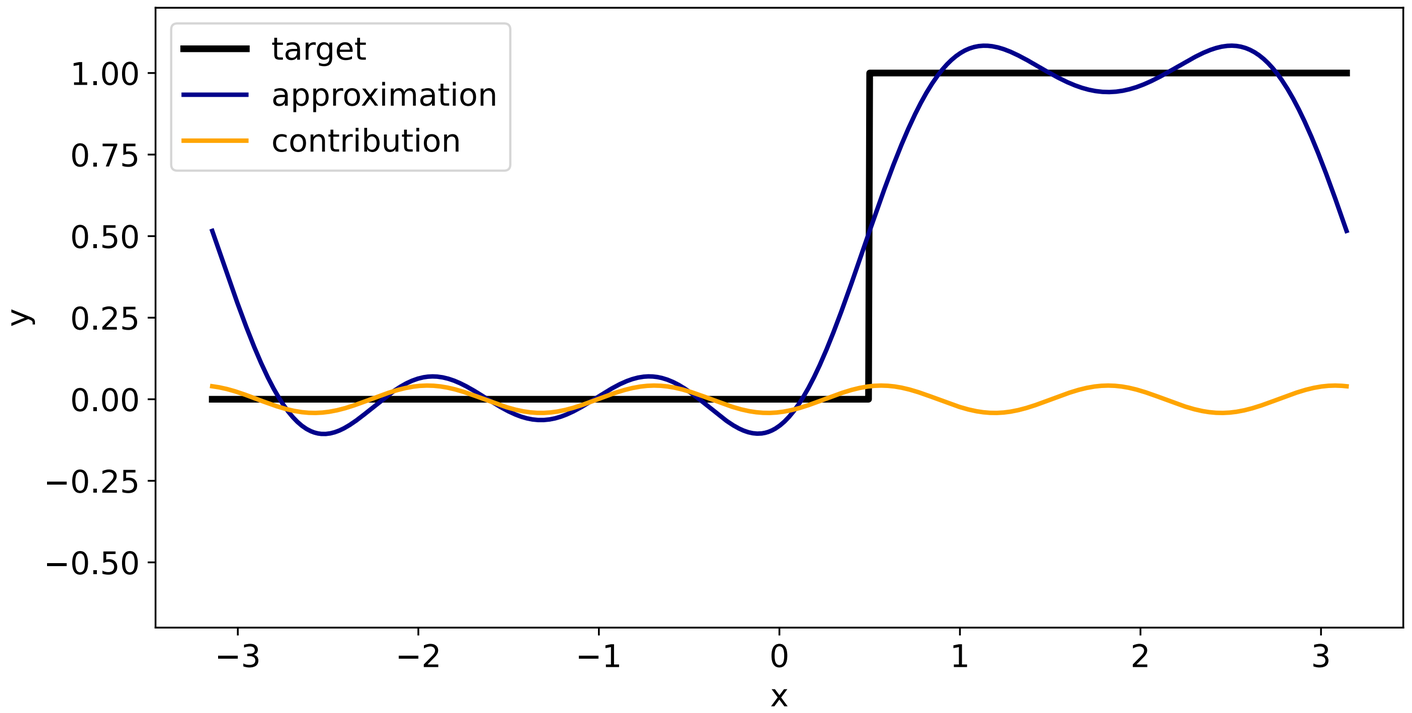


- One good way to represent functions is a Fourier basis:

$$f(\Delta) \approx \sum_{b=1}^{d} \alpha_b \sin(\theta_b \Delta) + \beta_b \cos(\theta_b \Delta) \qquad \Delta = j - i$$

  - with d different (fixed) frequencies θ_b, weights α_b, β_b

Representing functions: Fourier basis

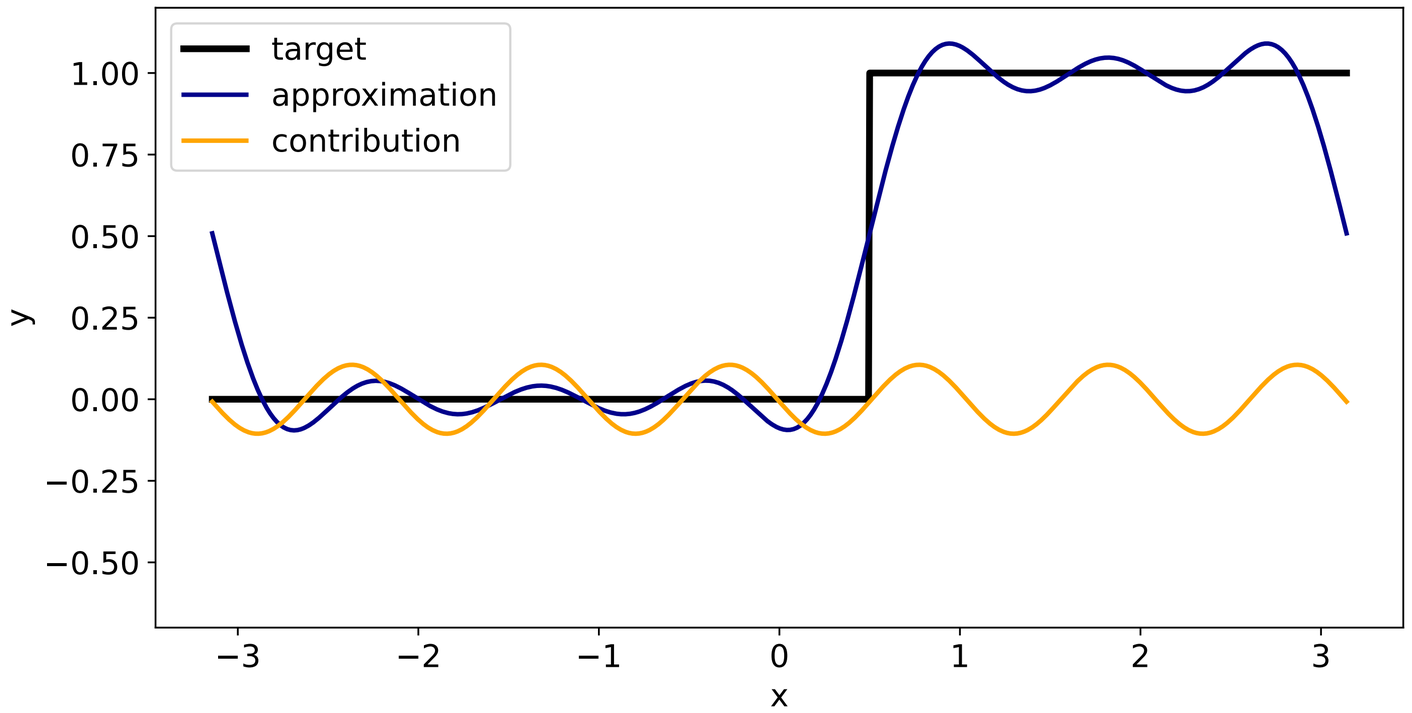


- One good way to represent functions is a Fourier basis:

$$f(\Delta) \approx \sum_{b=1}^{d} \alpha_b \sin(\theta_b \Delta) + \beta_b \cos(\theta_b \Delta) \qquad \Delta = j - i$$

  - with d different (fixed) frequencies θ_b, weights α_b, β_b

Representing functions: Fourier basis

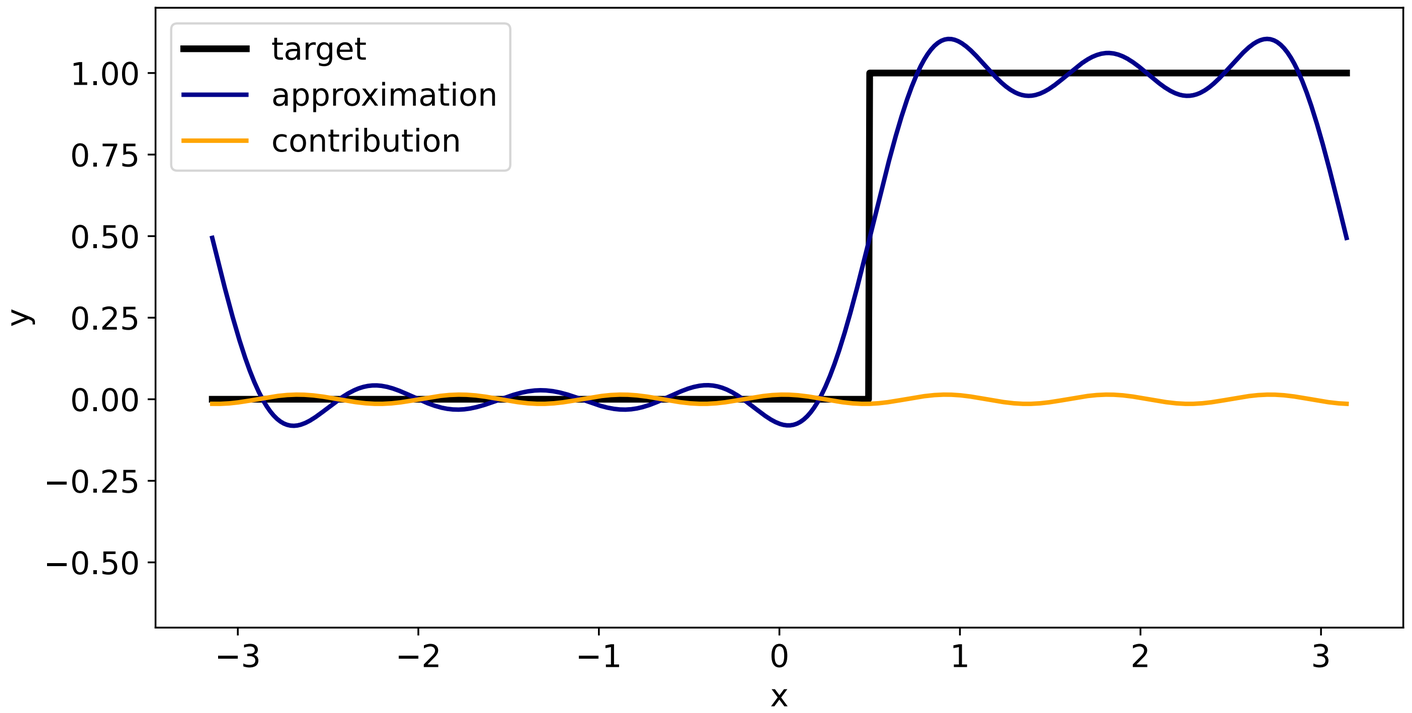


- One good way to represent functions is a Fourier basis:

$$f(\Delta) \approx \sum_{b=1}^{d} \alpha_b \sin(\theta_b \Delta) + \beta_b \cos(\theta_b \Delta) \qquad \Delta = j - i$$

  - with d different (fixed) frequencies θ_b, weights α_b, β_b

Representing functions: Fourier basis

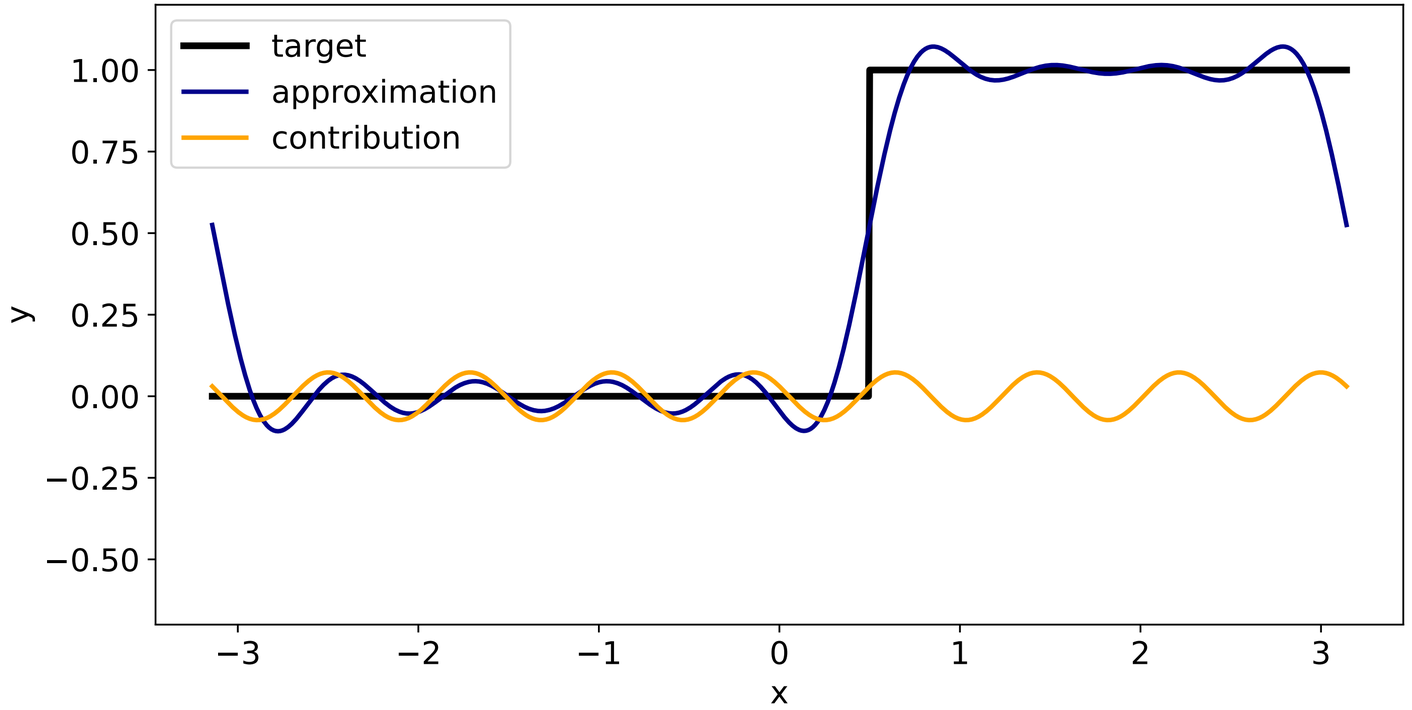


- One good way to represent functions is a Fourier basis:

$$f(\Delta) \approx \sum_{b=1}^{d} \alpha_b \sin(\theta_b \Delta) + \beta_b \cos(\theta_b \Delta) \qquad \Delta = j - i$$

  - with d different (fixed) frequencies θ_b, weights α_b, β_b

Representing functions: Fourier basis

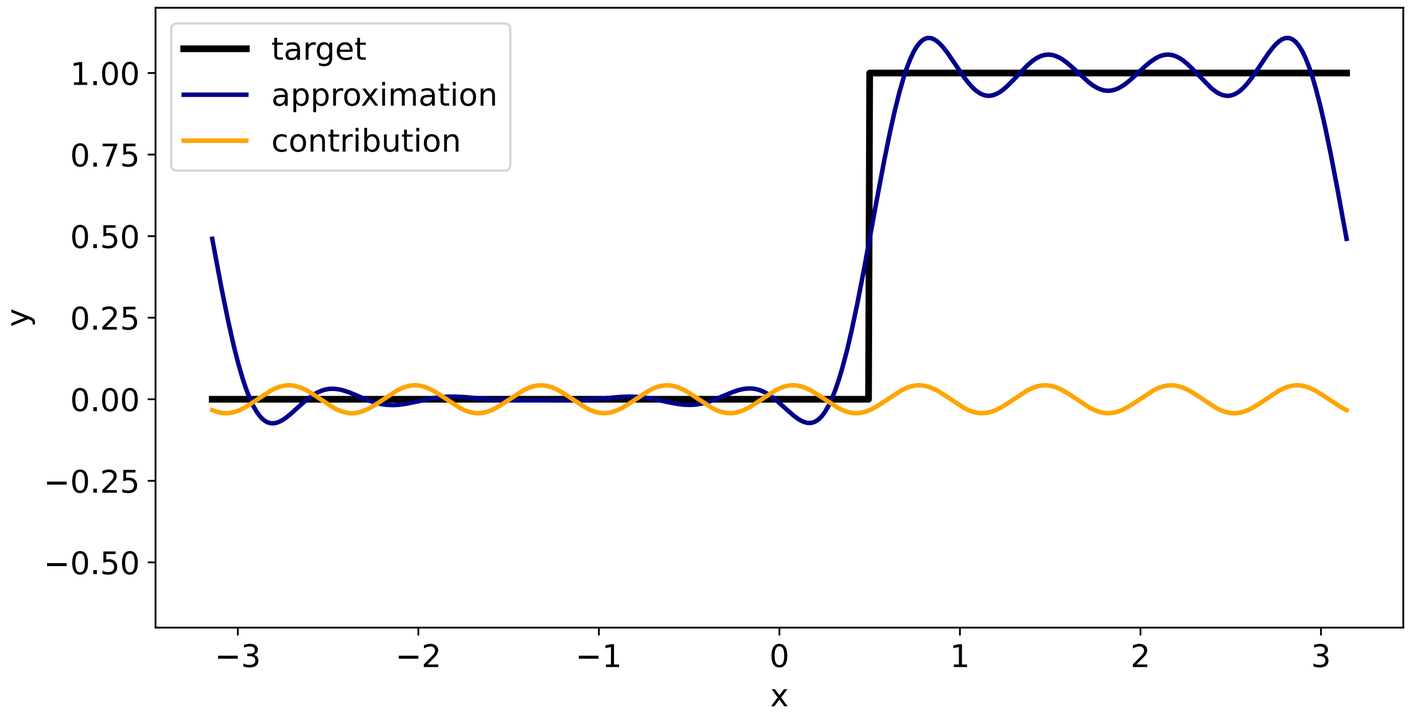


- One good way to represent functions is a Fourier basis:

$$f(\Delta) \approx \sum_{b=1}^{d} \alpha_b \sin(\theta_b \Delta) + \beta_b \cos(\theta_b \Delta) \qquad \Delta = j - i$$

  - with d different (fixed) frequencies θ_b, weights α_b, β_b

Representing functions: Fourier basis

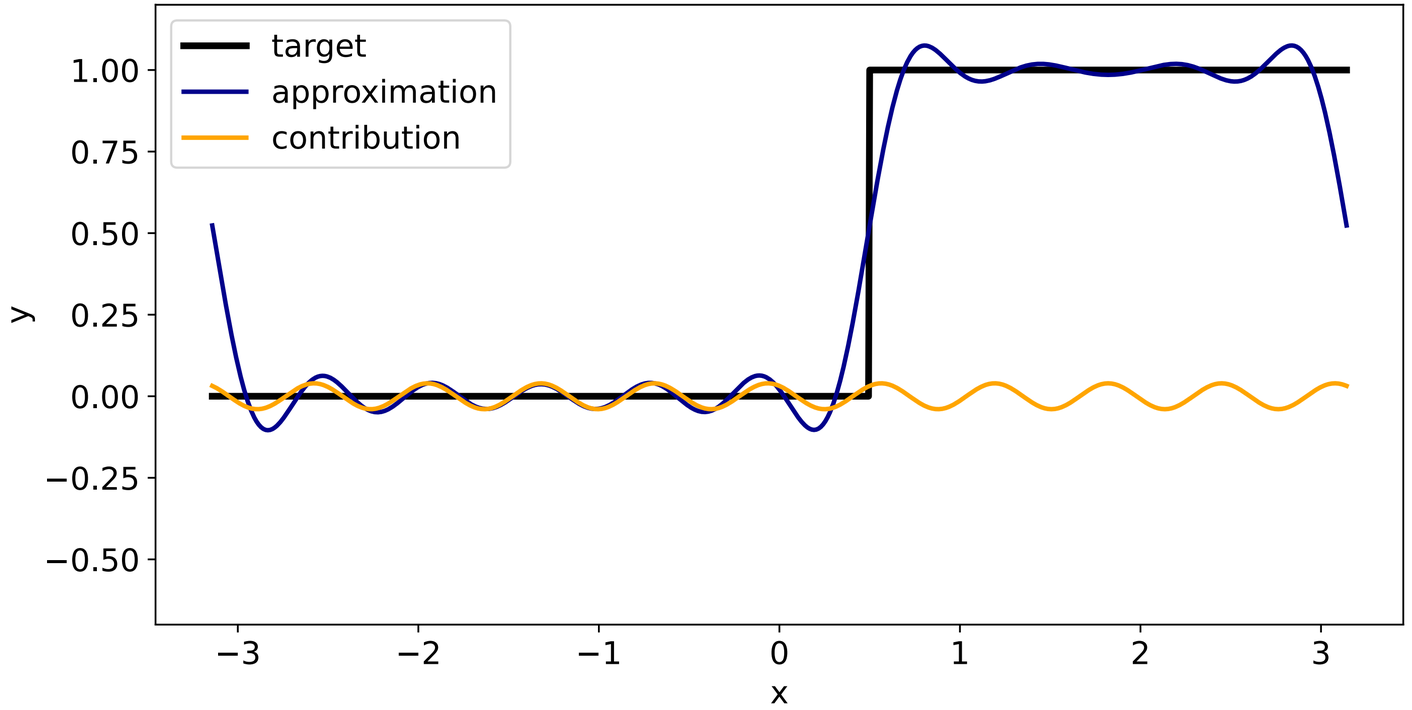


- One good way to represent functions is a Fourier basis:

$$f(\Delta) \approx \sum_{b=1}^{d} \alpha_b \sin(\theta_b \Delta) + \beta_b \cos(\theta_b \Delta) \qquad \Delta = j - i$$

  - with d different (fixed) frequencies θ_b, weights α_b, β_b

Representing functions: Fourier basis



- One good way to represent functions is a Fourier basis:

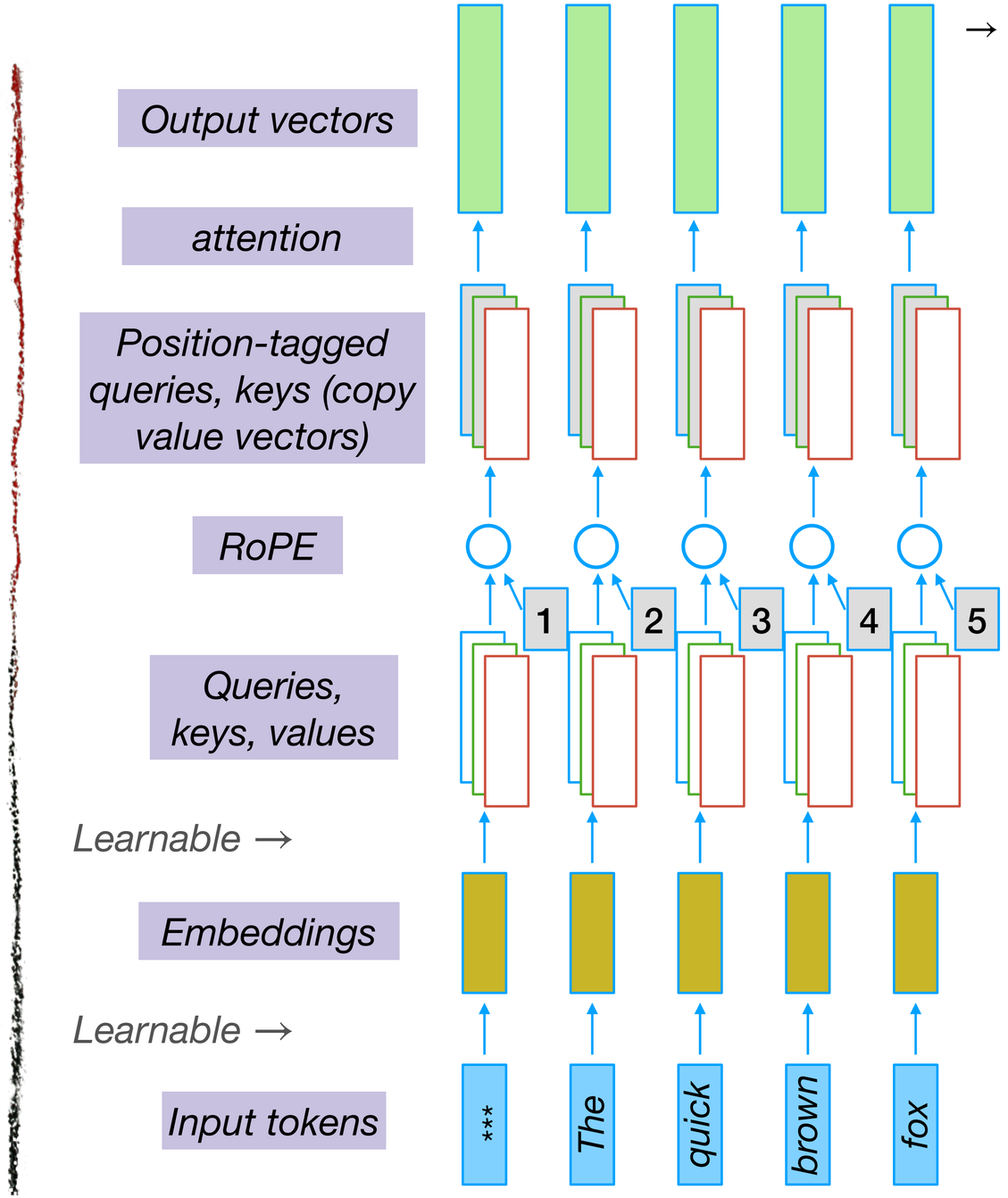
$$f(\Delta) \approx \sum_{b=1}^{d} \alpha_b \sin(\theta_b \Delta) + \beta_b \cos(\theta_b \Delta) \qquad \Delta = j - i$$

  - with d different (fixed) frequencies θ_b, weights α_b, β_b

Aside: why RoPE?

- Older *additive* style of positional encoding:
  - works at embeddings r.t. inside attention layer
  - [token embedding] + [position embedding]
  - same kind of Fourier basis for position
- RoPE improves inductive bias:
  - additive PE allows dependence on absolute position, has to learn to avoid this (results in extra noise, needs extra data)
  - RoPE lets content *gate* the contribution of position: e.g., by setting k to zero in all RoPE blocks, zero influence of position (but content can still contribute via non-RoPE blocks — or lowest frequencies can substitute)

Attention with RoPE

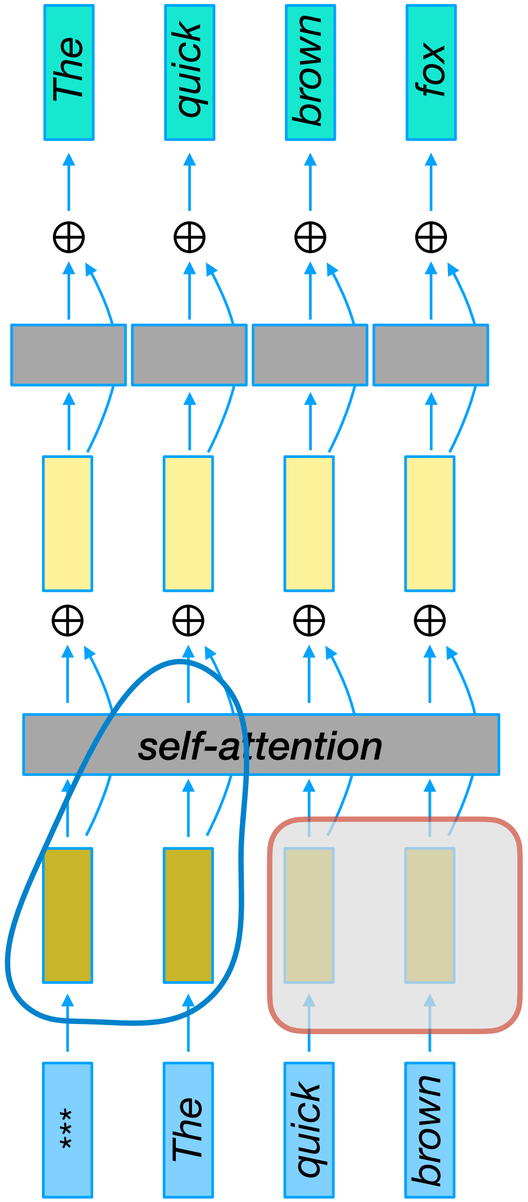


- Put it together: look up embedding for each input token; compute queries, keys, and values w/ weight matrices K, Q, V; rotate blocks in each query/key to encode position (token index); dot product, softmax, and sum for attention

Attention mask

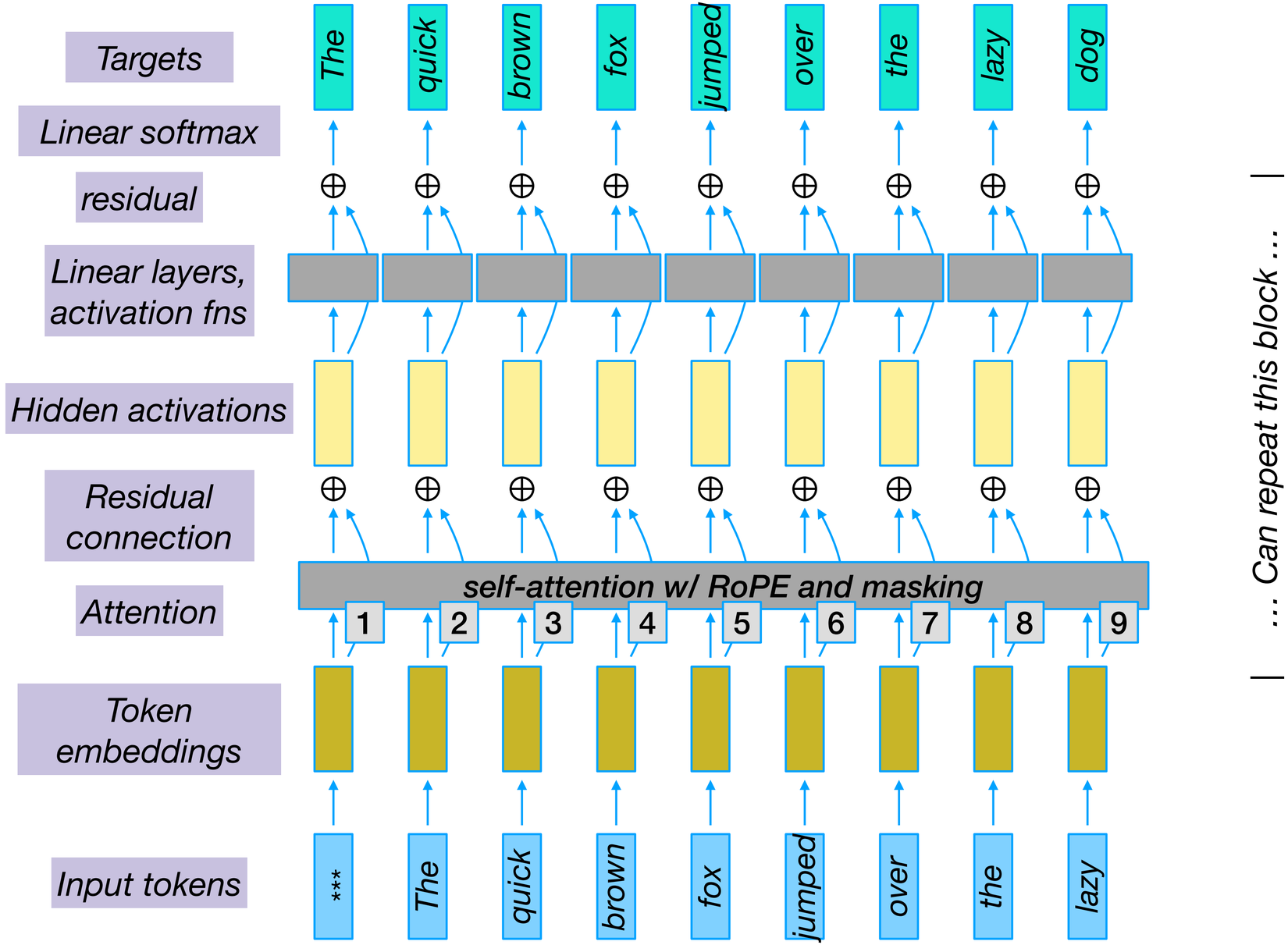
- Only let attention look at a subset of positions (*masked* attention)
- Here, own position and earlier in the sentence: causally masked self-attention

$$p_{ij} = \begin{cases} \dfrac{\exp(\frac{1}{\sqrt{d}} Q x_i \cdot K x_j)}{\sum_{j'=1}^{i} \exp(\frac{1}{\sqrt{d}} Q x_i \cdot K x_{j'})} & 1 \le j \le i \le T \\ 0 & \text{o/w} \end{cases}$$



Masked self-attention layers are perfect for next token prediction

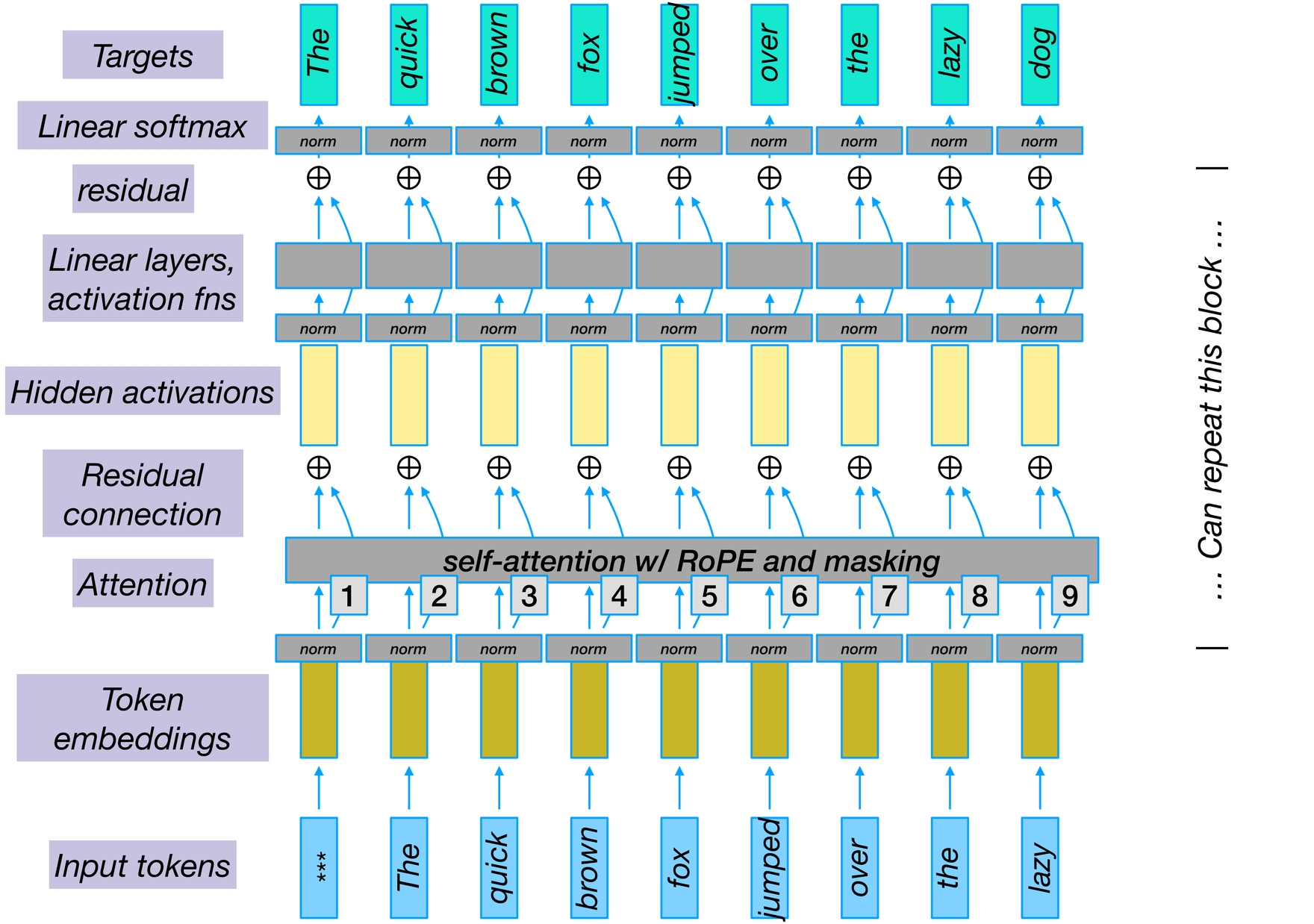
…with one more small modification



Transformer

This is LLaMA 3 if we use RMS norm, GQA, 2-layer MLP, SwiGLU

Original transformer used post-norm instead of pre-norm; positional encoding at embeddings



Transformer example

- Same tiny next-token problem as earlier: AABBA
  - one attention block with one head, 2d activations
- More complicated model than RNN, so the example leaves stuff out for simplicity
  - no norm, no pointwise MLP
- Instead of sweeping along sequence, we process entire sequence in parallel and sweep through depth

A tiny transformer on {A, B}*

one layer, one attention head

embed	$e_t = E\,\mathrm{onehot}(x_t)$
query, key	$\bar{q}_t = W_Q e_t, \quad \bar{k}_t = W_K e_t$
RoPE	$q_t = R(t\phi)\,\bar{q}_t, \quad k_t = R(t\phi)\,\bar{k}_t$
value	$v_t = W_V e_t$
attend	$\alpha_{tj} = \mathrm{softmax}_{j \le t}(q_t \cdot k_j)$
mix	$z_t = \sum_{j \le t} \alpha_{tj} v_j$
residual connection	$h_t = e_t + z_t$
read out	$o_t = U h_t + c$
predict	$p_t = \mathrm{softmax}(o_t)$
loss	$L = \sum_t -y_t \cdot \ln p_t, \quad y_t = \mathrm{onehot}(x_{t+1})$

$R(t\phi)$ rotates by $t\phi$: position t turns its query and key by $t \times 45°$.
The score $q_t \cdot k_j$ then depends on the positions only through the gap $t - j$.

Left out to fit on a slide: LayerNorm, the MLP block, multiple heads and layers.
W_O is folded into W_V, and the usual $1/\sqrt{d}$ score scaling into W_Q.

Parameters

E

1.00	0.00
0.00	1.00

e_A e_B

columns are the embeddings;
here simply the two axes

W_Q

1.00	4.00
-2.00	-2.00

W_K

1.00	1.00
0.00	0.00

W_V

0.00	0.00
1.00	-1.00

U

logit A	0.00	-1.00
logit B	0.00	1.00

c

1.00
0.00

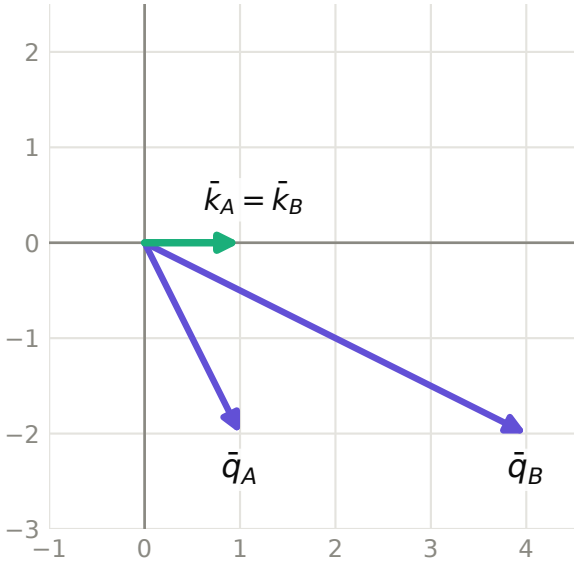
$\phi = 45°$

No positional vectors are added: position enters only through the rotation of queries and keys.

What the parameters do

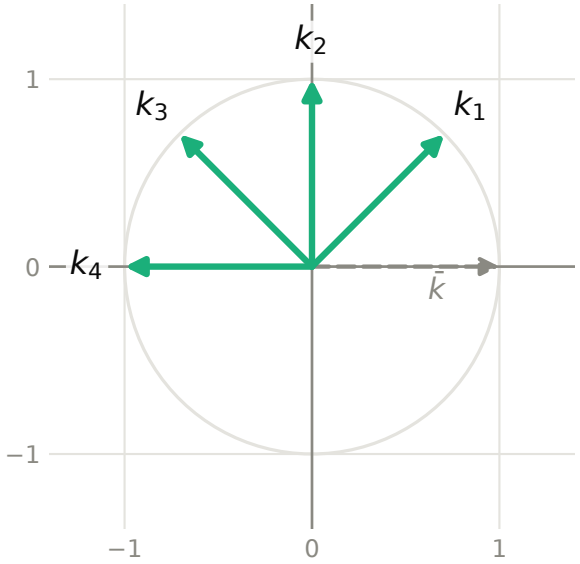
each panel shows one part of the head

Before rotation: W_Q, W_K



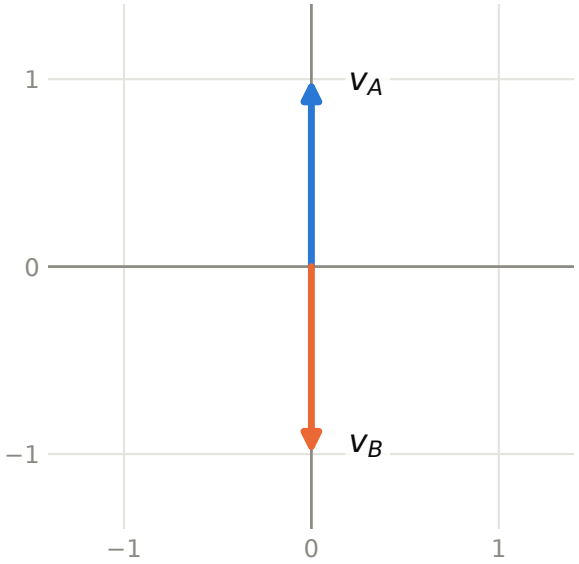
W_K sends both tokens to the same key, (1, 0): keys carry no token information. Both queries point down and to the right, about one rotation step (45°) behind the key direction.

RoPE: the key at position t



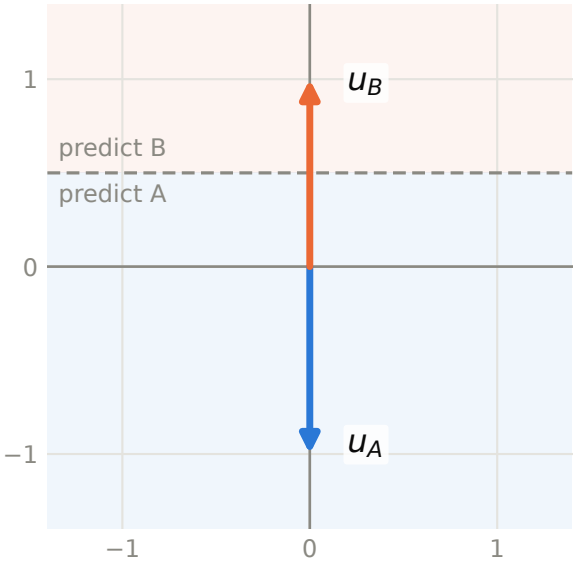
Position t turns its key by $t \times 45°$ (and its query by the same angle). A query that sits one step behind its own position lines up best with the previous key: a previous-token head.

Values: W_V



What each token contributes when attended to. v_A pushes coordinate 2 up, v_B pushes it down. Values depend only on the token.

Read-out: rows of U



Each logit is h_t dotted with a row of U, plus c. Dashed: where the two logits tie. Above it the model predicts B, below it A.

Forward 1: queries, keys and values

every position at once

inputs A A B B A — all four positions are processed at once; each predicts the next token (the last token is only a target)

1 2 3 4

queries

$\bar{q}_t = W_Q e_t, \quad q_t = R(t\phi)\,\bar{q}_t$

		e_t	W_Q	$\bar{q}_t$	rotate		q_t
1	A	1.00 0.00	→	1.00 -2.00	→	A 1φ = 45°	2.12 -0.71
2	A	1.00 0.00		1.00 -2.00		A 2φ = 90°	2.00 1.00
3	B	0.00 1.00		4.00 -2.00		B 3φ = 135°	-1.41 4.24
4	B	0.00 1.00		4.00 -2.00		B 4φ = 180°	-4.00 2.00

keys

$\bar{k}_t = W_K e_t, \quad k_t = R(t\phi)\,\bar{k}_t$

		e_t	W_K	$\bar{k}_t$	rotate		k_t
1	A	1.00 0.00	→	1.00 0.00	→	A 1φ = 45°	0.71 0.71
2	A	1.00 0.00		1.00 0.00		A 2φ = 90°	0.00 1.00
3	B	0.00 1.00		1.00 0.00		B 3φ = 135°	-0.71 0.71
4	B	0.00 1.00		1.00 0.00		B 4φ = 180°	-1.00 0.00

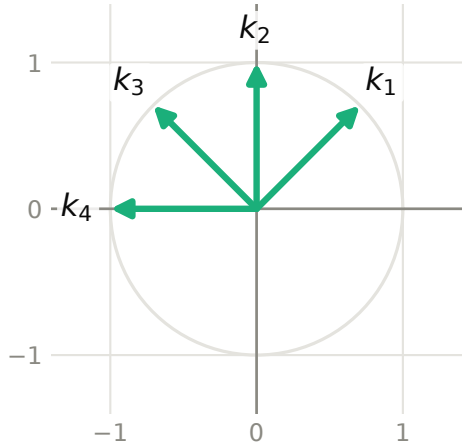
values

$v_t = W_V e_t$ (no rotation)

		e_t	W_V	v_t
1	A	1.00 0.00	→	0.00 1.00
2	A	1.00 0.00		0.00 1.00
3	B	0.00 1.00		0.00 -1.00
4	B	0.00 1.00		0.00 -1.00

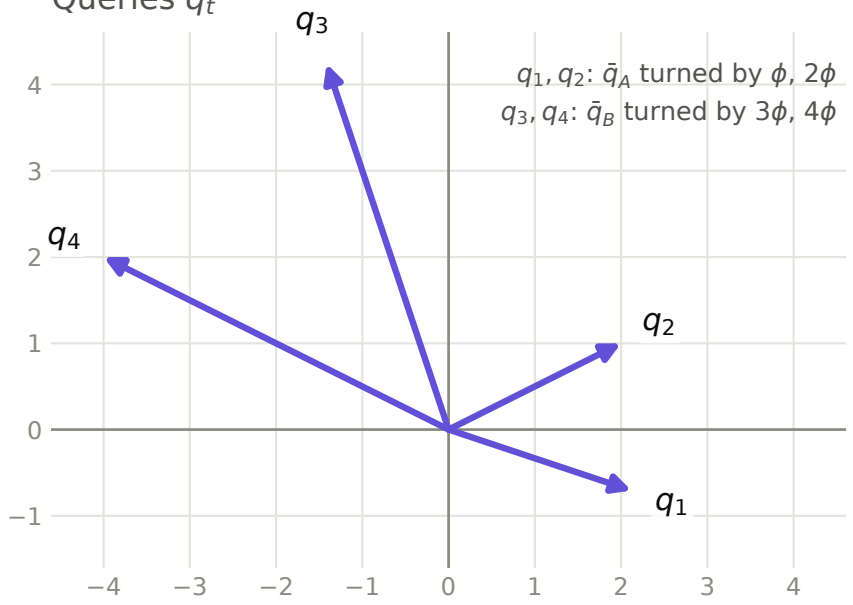
Values skip the rotation, so they depend only on the token: $v_1 = v_2 = v_A$ and $v_3 = v_4 = v_B$.

Keys k_t



All four keys are the same vector (1, 0), turned 45° per position.

Queries q_t



Forward 2: attention scores and weights

query position t looks at key positions $j \le t$

inputs **A** **A** **B** **B** **A**

1 2 3 4

all four positions are processed at once; each predicts the next token (the last token is only a target)

scores $q_t \cdot k_j$ (key position $j \rightarrow$)

query t		1	2	3	4
1	A	1.00			
2	A	2.12	1.00		
3	B	2.00	4.24	4.00	
4	B	-1.41	2.00	4.24	4.00

softmax each row →

weights α_{tj}

		1	2	3	4
1	A	1.00			
2	A	0.75	0.25		
3	B	0.06	0.53	0.41	
4	B	0.00	0.06	0.53	0.41

Grey cells are masked: a position cannot look ahead. Every row's largest weight sits one step left of the diagonal, on the previous token. The rest goes mostly to the position itself.

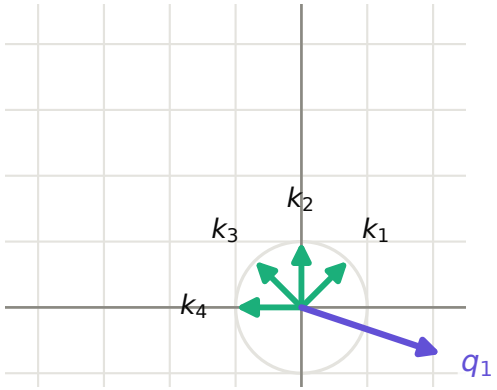
Row 4, worked out:

$\alpha_{4,\cdot} = \text{softmax}(-1.41,\ 2.00,\ 4.24,\ 4.00)$

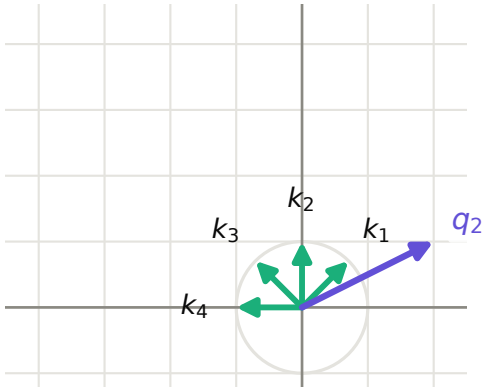
$= (0.00, 0.06, 0.53, 0.41)$

e.g. $q_4 \cdot k_3$ = (-4.00)(-0.71) + (2.00)(0.71) = 4.24

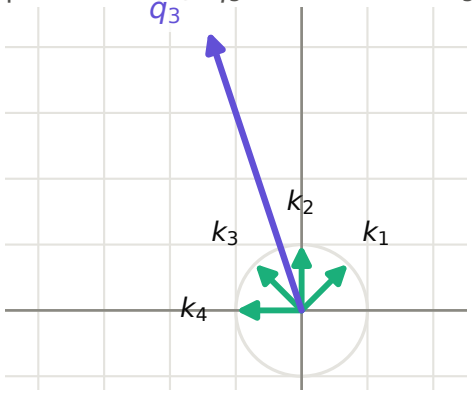
position 1 (A): q_1 is 63° behind k_1



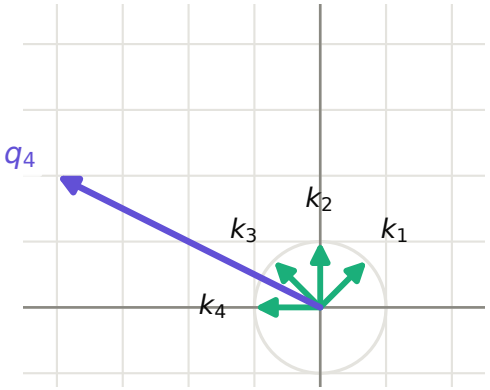
position 2 (A): q_2 is 63° behind k_2



position 3 (B): q_3 is 27° behind k_3



position 4 (B): q_4 is 27° behind k_4



Drawn to scale. Every query trails its own key, so it sits 18° from the previous position's key (45° behind) and scores that key highest. The B queries are twice as long as the A queries, so their scores spread further apart and their weights are sharper.

Forward 3: mixing the values

$$z_t = \sum_{j \le t} \alpha_{tj} v_j \text{ for every position}$$

inputs **A** **A** **B** **B** **A**

1 2 3 4

all four positions are processed at once; each predicts the next token (the last token is only a target)

weights α_{tj} (row t, column j)

		1	2	3	4
1	A	1.00			
2	A	0.75	0.25		
3	B	0.06	0.53	0.41	
4	B	0.00	0.06	0.53	0.41

×

values v_j (row j)

1 A	0.00	1.00
2 A	0.00	1.00
3 B	0.00	-1.00
4 B	0.00	-1.00

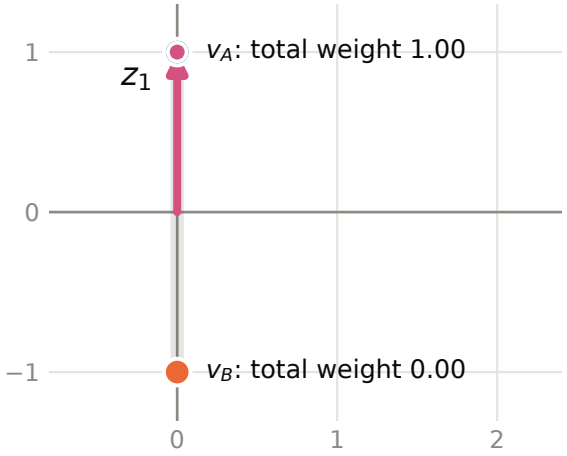
=

z_t (row t)

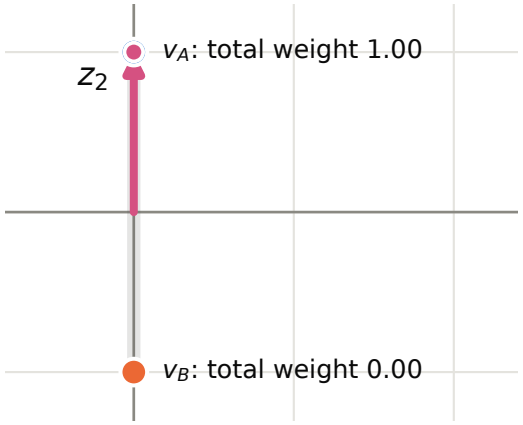
1 A	0.00	1.00
2 A	0.00	1.00
3 B	0.00	0.17
4 B	0.00	-0.88

Row t of the weights says how much of each value position t takes. Because v_j is either v_A or v_B, only the total weight on A positions versus B positions matters.

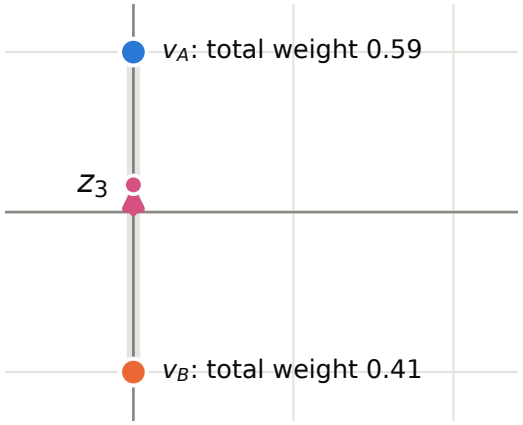
position 1 (A): $z_1 = (0.00, 1.00)$



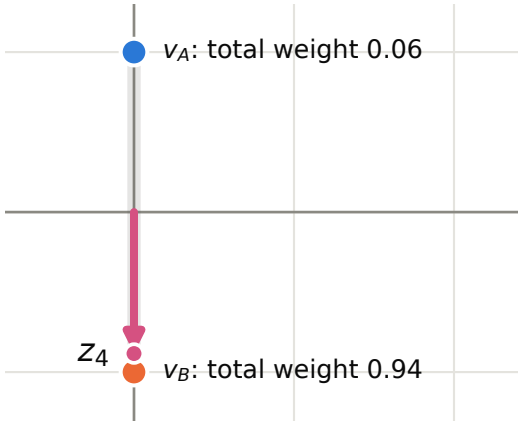
position 2 (A): $z_2 = (0.00, 1.00)$



position 3 (B): $z_3 = (0.00, 0.17)$



position 4 (B): $z_4 = (0.00, -0.88)$



Total weight on v_A: the sum of row t's weights over the positions holding A (likewise for B). It sets where z_t lands on the grey segment.
The next slide follows position 4 step by step.

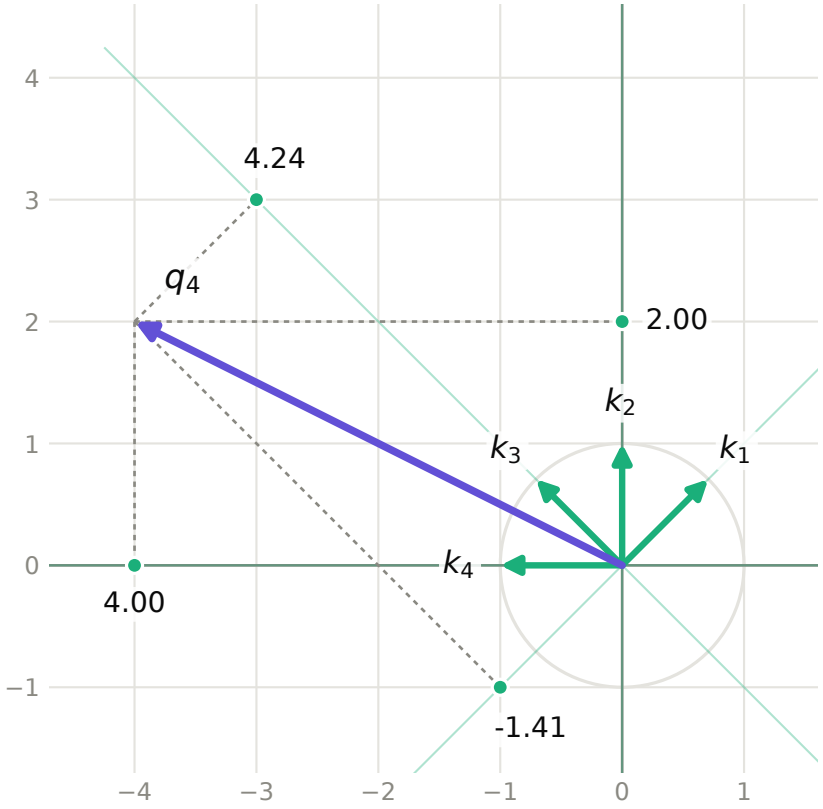
Forward 4: a closer look at position 4

query q_4 against keys $k_1, \ldots, k_4$, then a mix of values

inputs A A B B A — position 4 reads B and must predict A

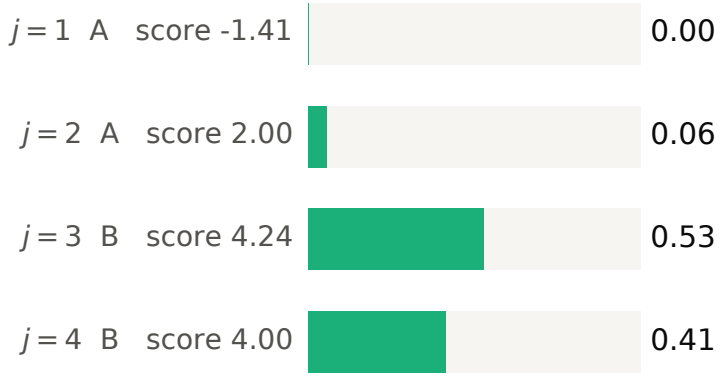
1 2 3 4

1. Scores: project q_4 onto each key

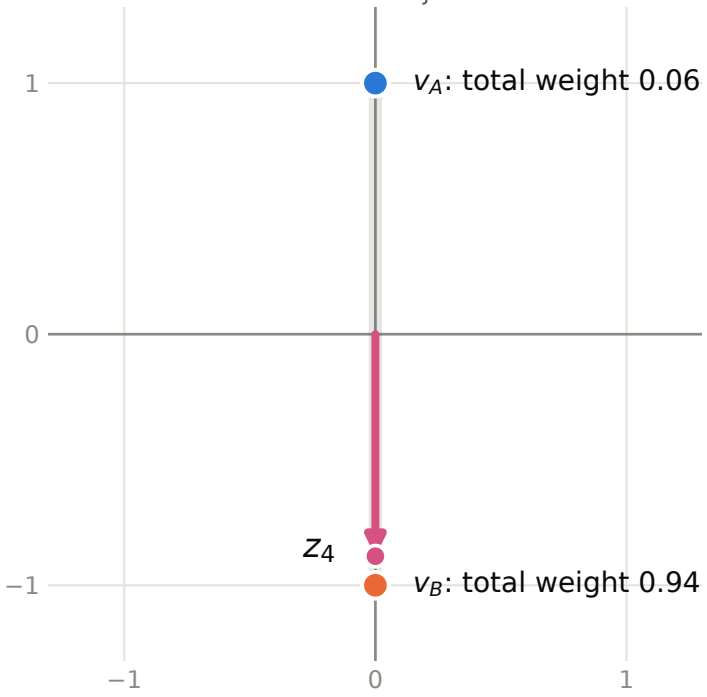


Each key has length 1, so $q_4 \cdot k_j$ is how far along k_j's line the dotted drop from q_4 lands (negative: behind the origin).

2. Weights α_{4j} = softmax of the scores



3. Mix the values: $z_4 = \sum_j \alpha_{4j} v_j$



Total weight: positions 1, 2 (A) get 0.06; 3, 4 (B) get 0.94.
z_4 = 0.06 v_A + 0.94 v_B = (0.00, -0.88).
Values depend only on the token, so every z_t lies on the grey segment.

Forward 5: residual connection, read-out and loss

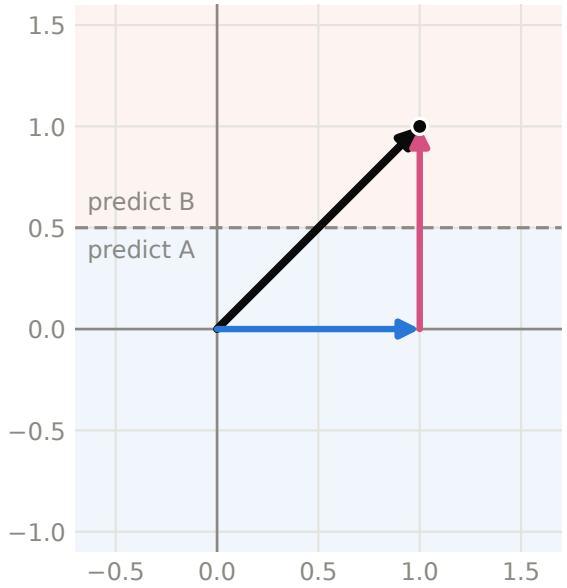
total loss L = 2.24

inputs A A B B A — all four positions are processed at once; each predicts the next token (the last token is only a target)

	e_t		z_t		h_t		$o_t = Uh_t + c$		p_t (A, B)		$L_t = -\ln p_t[x_{t+1}]$
1 A	1.00	0.00	0.00	1.00	1.00	1.00	0.00	1.00	0.27	0.73	1.31
2 A	1.00	0.00	0.00	1.00	1.00	1.00	0.00	1.00	0.27	0.73	0.31
3 B	0.00	1.00	0.00	0.17	0.00	1.17	-0.17	1.17	0.21	0.79	0.23
4 B	0.00	1.00	0.00	-0.88	0.00	0.12	0.88	0.12	0.68	0.32	0.38

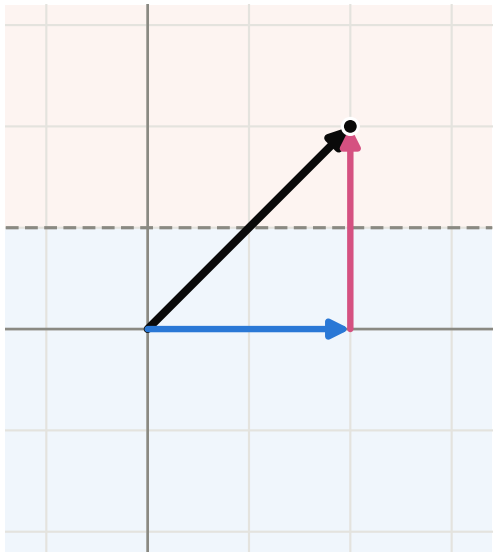
Each position predicts its own next token; the losses add up.

t = 1, reads A



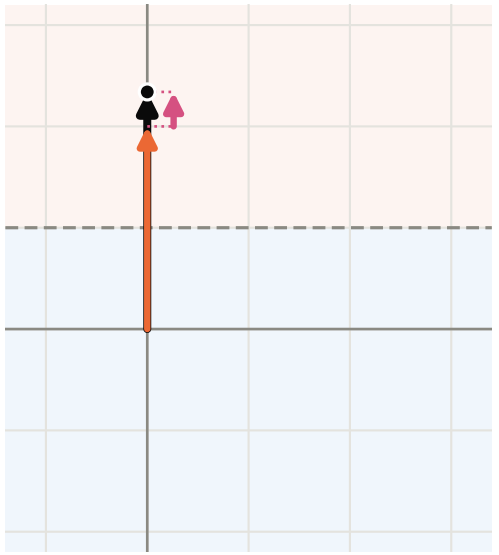
p(A) 0.27 p(B) 0.73 target A L_1 = 1.31

t = 2, reads A



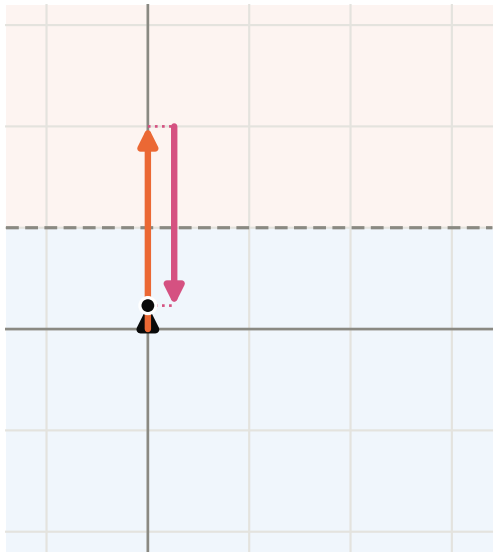
p(A) 0.27 p(B) 0.73 target B L_2 = 0.31

t = 3, reads B



p(A) 0.21 p(B) 0.79 target B L_3 = 0.23

t = 4, reads B



p(A) 0.68 p(B) 0.32 target A L_4 = 0.38

e_t (token colour) $+ z_t$ from attention $= h_t$

Shading: the read-out's prediction. At t = 1 attention can only return v_A.

Backward 1: from the loss to the residual stream

$g_t = \partial L / \partial h_t$ at every position

inputs **A** **A** **B** **B** **A**

1 2 3 4

the backward pass runs the forward steps in reverse, all positions at once

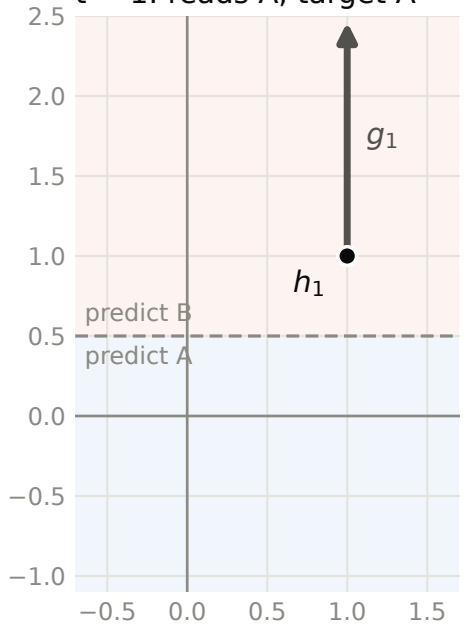
		p_t (A, B)			y_t (target)		$\partial L/\partial o_t$		$g_t = U^\top (p_t - y_t)$	
1	A	0.27	0.73	−	1	0	-0.73	0.73	0.00	1.46
2	A	0.27	0.73		0	1	0.27	-0.27	0.00	-0.54
3	B	0.21	0.79		0	1	0.21	-0.21	0.00	-0.41
4	B	0.68	0.32		1	0	-0.32	0.32	0.00	0.63

$U^\top$ ⟶

$\partial L_t/\partial p_t = -y_t/p_t$, and back through the softmax this becomes $p_t - y_t$. Each g_t is a combination of the rows of U: $g_t = (p_{t,A} - y_{t,A})\, u_A + (p_{t,B} - y_{t,B})\, u_B$.

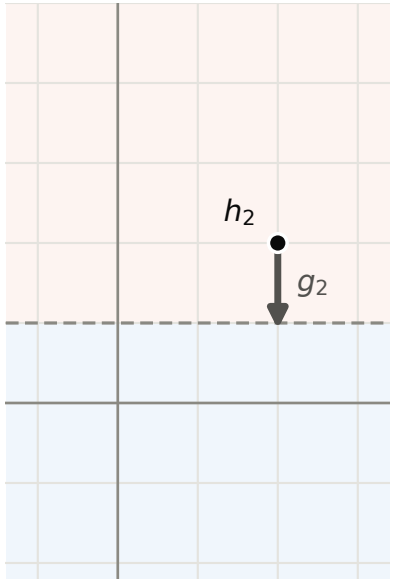
Residual connection $h_t = e_t + z_t$: the same g_t is passed to both branches: straight to $\partial L/\partial e_t$ (the residual route), and into attention as $\partial L/\partial z_t$.

t = 1: reads A, target A



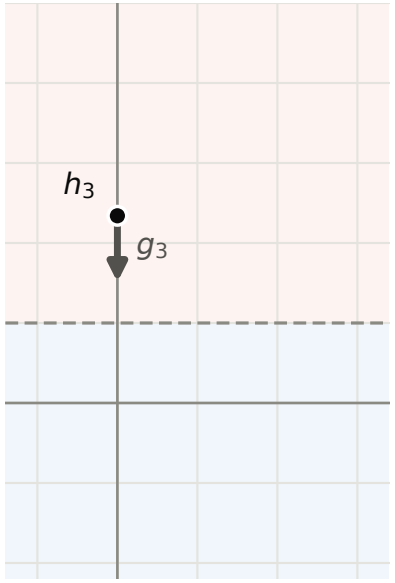
p_1[A] = 0.27
L_1 = 1.31

t = 2: reads A, target B



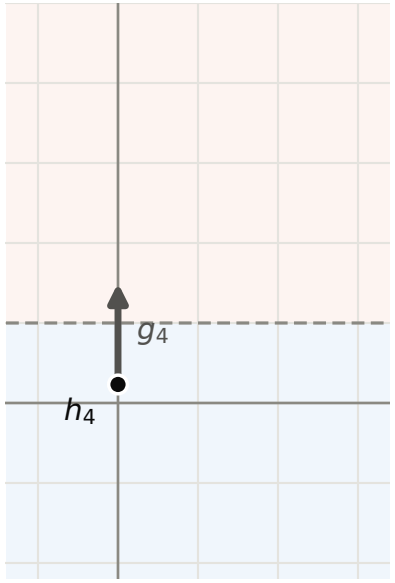
p_2[B] = 0.73
L_2 = 0.31

t = 3: reads B, target B



p_3[B] = 0.79
L_3 = 0.23

t = 4: reads B, target A



p_4[A] = 0.68
L_4 = 0.38

Each g_t is drawn starting at h_t. It points away from the target's region, so a gradient-descent step would move h_t toward it. The worse the prediction, the longer the arrow.

Backward 2: back through attention's value vectors

inputs A A B B A

1 2 3 4

$z_t = \sum_j \alpha_{tj} v_j$ and $\partial L/\partial z_t = g_t$, so value j collects gradient from every position that read it

weights α_{tj}: read column j

		1	2	3	4
1	A	1.00			
2	A	0.75	0.25		
3	B	0.06	0.53	0.41	
4	B	0.00	0.06	0.53	0.41

(rows: position t)

and g_t

1 A	0.00	1.46
2 A	0.00	-0.54
3 B	0.00	-0.41
4 B	0.00	0.63

→ $\partial L/\partial v_j = \sum_{t \geq j} \alpha_{tj} g_t$

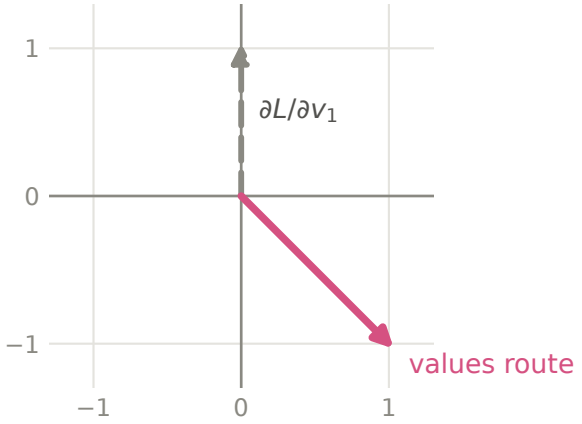
1 A	0.00	1.03
2 A	0.00	-0.32
3 B	0.00	0.16
4 B	0.00	0.26

$W_V^\top$ ⟶ values route to e_j

1.03	-1.03
-0.32	0.32
0.16	-0.16
0.26	-0.26

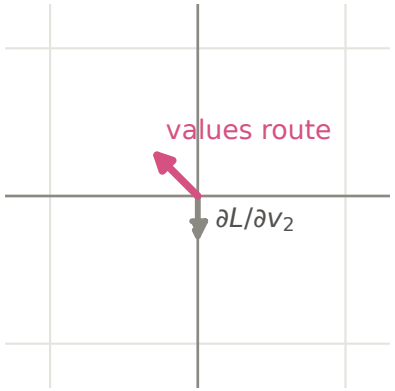
Value j is used by every position $t \geq j$ that attends to it, so its gradient is a weighted sum of their g_t, weighted by the same α_{tj}.

value 1 (A)



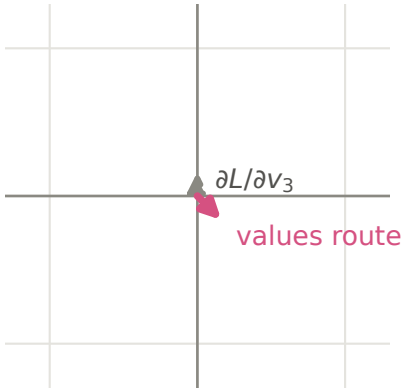
$\partial L/\partial v_1 = 1.00\ g_1 + 0.75\ g_2 + 0.06\ g_3 + 0.00\ g_4$

value 2 (A)



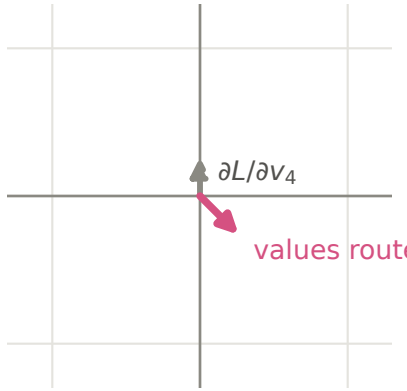
$\partial L/\partial v_2 = 0.25\ g_2 + 0.53\ g_3 + 0.06\ g_4$

value 3 (B)



$\partial L/\partial v_3 = 0.41\ g_3 + 0.53\ g_4$

value 4 (B)



$\partial L/\partial v_4 = 0.41\ g_4$

Dashed: $\partial L/\partial v_j$ (always vertical, because every g_t is). Solid: after $W_V^\top$, the values route to e_j. W_V only reads $e_1 - e_2$, so this route always points along $(1, -1)$.

Backward 3: back through the softmax to the scores

one number per (query, key) pair

inputs A A B B A

z_t also depends on the weights α_{tj}, and they depend on the scores $s_{tj} = q_t \cdot k_j$

$\partial L/\partial \alpha_{tj} = g_t \cdot v_j$

		1	2	3	4
1	A	1.46			
2	A	-0.54	-0.54		
3	B	-0.41	-0.41	0.41	
4	B	0.63	0.63	-0.63	-0.63

softmax backward →

$\partial L/\partial s_{tj} = \alpha_{tj}\,(\partial L/\partial \alpha_{tj} - \sum_k \alpha_{tk}\,\partial L/\partial \alpha_{tk})$

		1	2	3	4
1	A	0.00			
2	A	0.00	0.00		
3	B	-0.02	-0.18	0.20	
4	B	0.00	0.07	-0.04	-0.03

Positive: raising that score would raise the loss.
Negative: raising it would lower the loss.
Each row sums to zero, because the weights always sum to 1.

Row 3, worked out

$g_3 =$ (0.00, -0.41): the loss at position 3 falls if h_3 moves up (toward predicting B).
$\partial L/\partial \alpha_{3j} = g_3 \cdot v_j$ = -0.41, -0.41, 0.41 (the A values push h_3 up, which helps)

weighted average $\sum_k \alpha_{3k}\,\partial L/\partial \alpha_{3k}$ = -0.07; $\partial L/\partial s_{3j} = \alpha_{3j} \times$ (difference) = -0.02, -0.18, 0.20

So the gradient asks for more weight on key 2 (an A) and less on key 3 (a B). Key 1 is also an A, but it has little weight to begin with, so its score gradient is small: the softmax scales each entry by α_{tj}.

Rows 1 and 2 are all zeros

Positions 1 and 2 can only see A tokens, and every A has the same value v_A (values are not rotated). Moving their attention around cannot change z_1 or z_2, so their scores get no gradient, and neither do queries 1 and 2.

Backward 4: back through RoPE to the queries and keys

$s_{tj} = q_t \cdot k_j$

inputs A A B B A — gradients of a dot product: $\partial s_{tj}/\partial q_t = k_j$ and $\partial s_{tj}/\partial k_j = q_t$

1 2 3 4

queries

$$\partial L/\partial q_t = \sum_j \partial L/\partial s_{tj}\, k_j$$

$\partial L/\partial q_t$

1 A	0.00	0.00
2 A	0.00	0.00
3 B	-0.16	-0.05
4 B	0.06	0.04

rotate back →

1 A	$-1\varphi = -45°$	
2 A	$-2\varphi = -90°$	
3 B	$-3\varphi = -135°$	
4 B	$-4\varphi = -180°$	

$\partial L/\partial \bar{q}_t$

0.00	0.00
0.00	0.00
0.07	0.15
-0.06	-0.04

$W_Q^\top$ →

queries route to e_t

0.00	0.00
0.00	0.00
-0.22	-0.01
0.02	-0.16

keys

$$\partial L/\partial k_j = \sum_t \partial L/\partial s_{tj}\, q_t$$

$\partial L/\partial k_j$

1 A	0.02	-0.08
2 A	-0.01	-0.64
3 B	-0.13	0.78
4 B	0.12	-0.06

rotate back →

1 A	$-1\varphi = -45°$	
2 A	$-2\varphi = -90°$	
3 B	$-3\varphi = -135°$	
4 B	$-4\varphi = -180°$	

$\partial L/\partial \bar{k}_j$

-0.04	-0.07
-0.64	0.01
0.64	-0.46
-0.12	0.06

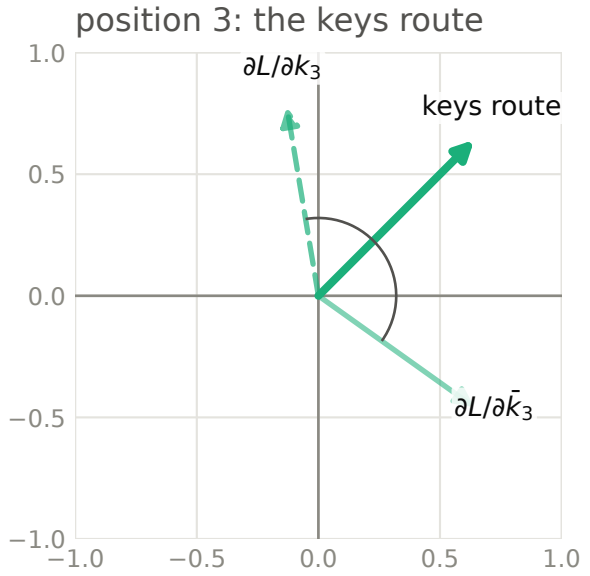
$W_K^\top$ →

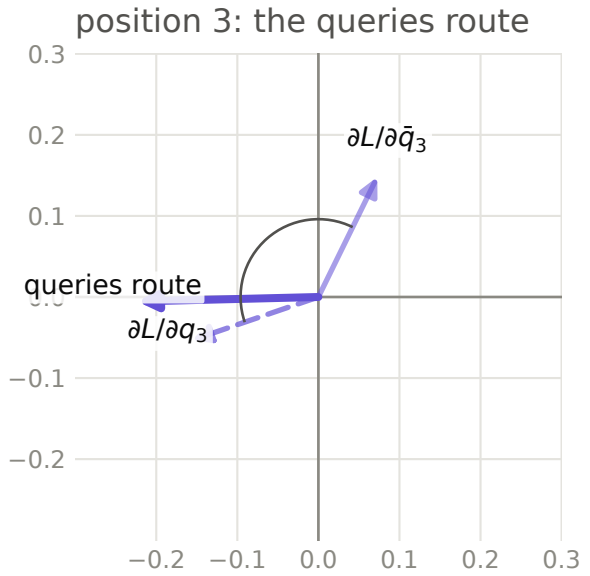
keys route to e_t

-0.04	-0.04
-0.64	-0.64
0.64	0.64
-0.12	-0.12

Rotating back by $-t\varphi$ applies $R(t\varphi)^\top$, the transpose of the forward rotation.
Every key is $W_K e$ with W_K reading only $e_1 + e_2$, so every keys route points along (1, 1).

Right: position 3, where attention matters most. Dashed: the gradient at the rotated vector.
Faint: the same gradient rotated back by −135°. Solid: the keys or queries route to e_3, after $W_K^\top$ or $W_Q^\top$.
The two plots use different scales.





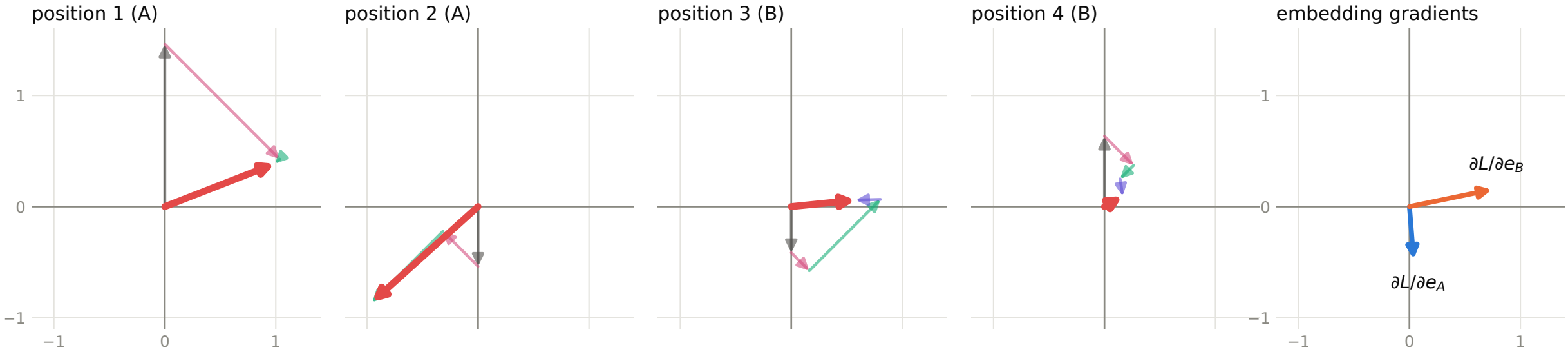
Backward 5: adding up the four routes

the four routes to e_t, added tip to tail

inputs **A** **A** **B** **B** **A** all four positions are processed at once; each predicts the next token (the last token is only a target)

		residual route		+ values route		+ keys route		+ queries route		= $\partial L/\partial e_t$	
1	A	0.00	1.46	1.03	-1.03	-0.04	-0.04	0.00	0.00	0.99	0.39
2	A	0.00	-0.54	-0.32	0.32	-0.64	-0.64	0.00	0.00	-0.96	-0.86
3	B	0.00	-0.41	0.16	-0.16	0.64	0.64	-0.22	-0.01	0.58	0.06
4	B	0.00	0.63	0.26	-0.26	-0.12	-0.12	0.02	-0.16	0.16	0.09

The queries route is zero at positions 1 and 2 (see Backward 3). Position 1's keys route is tiny: only query 3 cares about key 1, and only a little.



$\partial L/\partial e_A$ = pos. 1 + 2 = (0.04, -0.47)

$\partial L/\partial e_B$ = pos. 3 + 4 = (0.74, 0.15)

residual route · values route · keys route · queries route · total $\partial L/\partial e_t$ (added tip to tail in this order)

Weight-matrix gradients (not drawn)

computable from quantities on the earlier slides

To keep the pictures simple, these slides draw only gradients with respect to vectors. Every weight gradient follows one rule:

for a linear map $y_t = Wx_t$ used at every position t: $$\frac{\partial L}{\partial W} = \sum_t \frac{\partial L}{\partial y_t} x_t^\top$$

a sum of outer products: (gradient on the output) times (input), transposed. A bias is the same rule with input 1.

weights	linear map	gradient	where the pieces are shown
$U,\ c$	$o_t = Uh_t + c$	$\sum_t (p_t - y_t)\, h_t^\top,\quad \sum_t (p_t - y_t)$	$p_t - y_t$: Backward 1; h_t: Forward 5
W_V	$v_j = W_V e_j$	$\sum_j \frac{\partial L}{\partial v_j} e_j^\top$	$\partial L/\partial v_j$: Backward 2
W_Q	$\bar{q}_t = W_Q e_t$	$\sum_t \frac{\partial L}{\partial \bar{q}_t} e_t^\top$	$\partial L/\partial \bar{q}_t$ (after rotating back): Backward 4
W_K	$\bar{k}_j = W_K e_j$	$\sum_j \frac{\partial L}{\partial \bar{k}_j} e_j^\top$	$\partial L/\partial \bar{k}_j$ (after rotating back): Backward 4
E	$e_t = E\,\mathrm{onehot}(x_t)$	$\sum_t \frac{\partial L}{\partial e_t} \mathrm{onehot}(x_t)^\top$	column A = sum over positions holding A: Backward 5

The gradient for W_Q and W_K uses the gradient before the rotation, because W_Q and W_K act before RoPE. The angle φ is fixed, so it has no gradient.

Each weight receives contributions from every position that uses it, just as each value vector collected gradient from every position that read it.

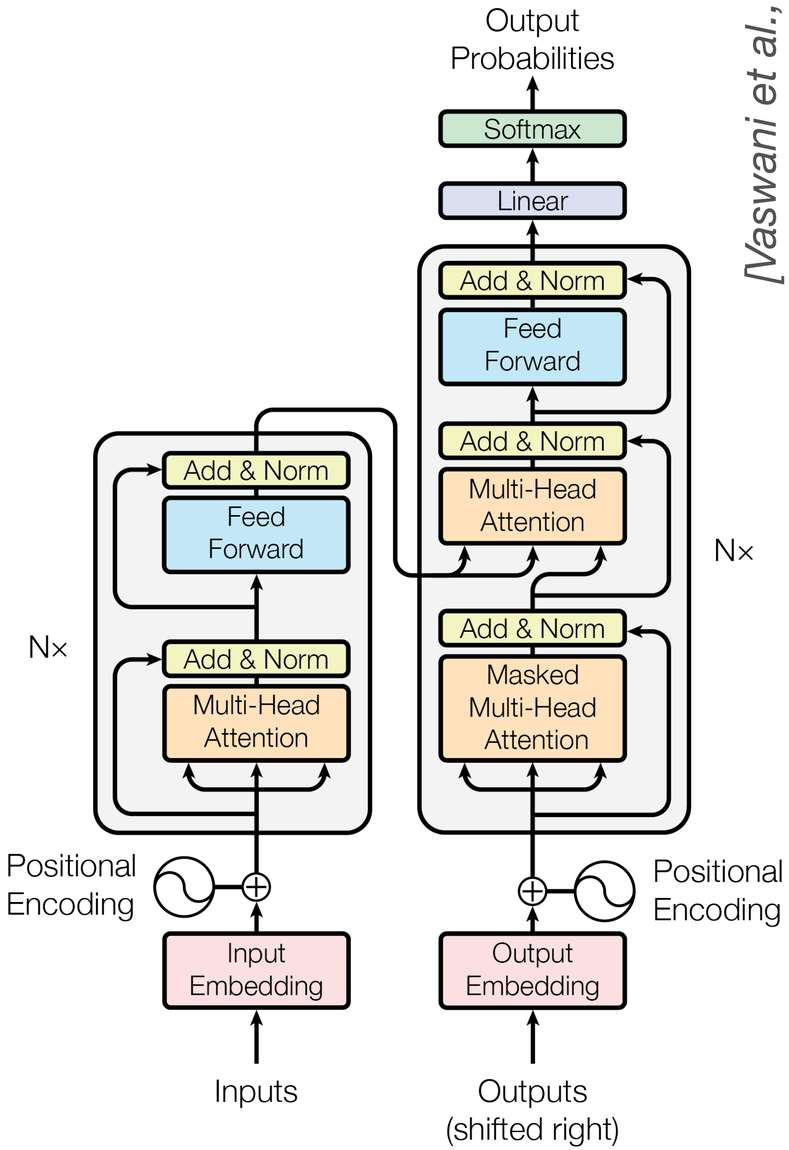
Transduction, generation, classification

- Above we designed a transformer for a *transduction* task (each input has a corresponding output)
  - then used it for *generation* by masking, shifting tokens after each step and appending outputs
- For a many:1 task like classification, we can add a distinguished token *** at beginning or end, read result off of corresponding output token
  - turn off masking
  - beginning and end are symmetric, doesn't matter which

Encoder-decoder

not as common any more, since we can just increase context length instead

- If we have side info that we always want available to attend to, can use a two-part network:
- Encoder looks at side info, outputs a DB of embeddings
  - can use unmasked (bidirectional) attention
- Decoder attends over encoder's DB plus its own (backprop through both)
  - unmasked attention to encoder
  - decoder can use masking and shifting for completion
- Previous architecture is often called *decoder-only* since it is analogous to decoder half



[Vaswani et al., 2017]

Why transformers?

- Flexible: variable-length inputs, multimodal data, enables large-scale search (e.g., RAG)
- Capable: handle a wide range of ML problems well
  - this wasn't a priori obvious: e.g., can construct reasonable problems where RNNs provably do better
- Long context: don't forget distant past even while being accurate and detailed in the short term (cf. RNNs)
  - lets us provide, produce, and benefit from lots of info at once: extra examples, rules/policies, complex reasoning
- Scaling laws
  - experiments on small models let us design successful larger models
  - e.g., tune hyperparameters, test model features

Why transformers?

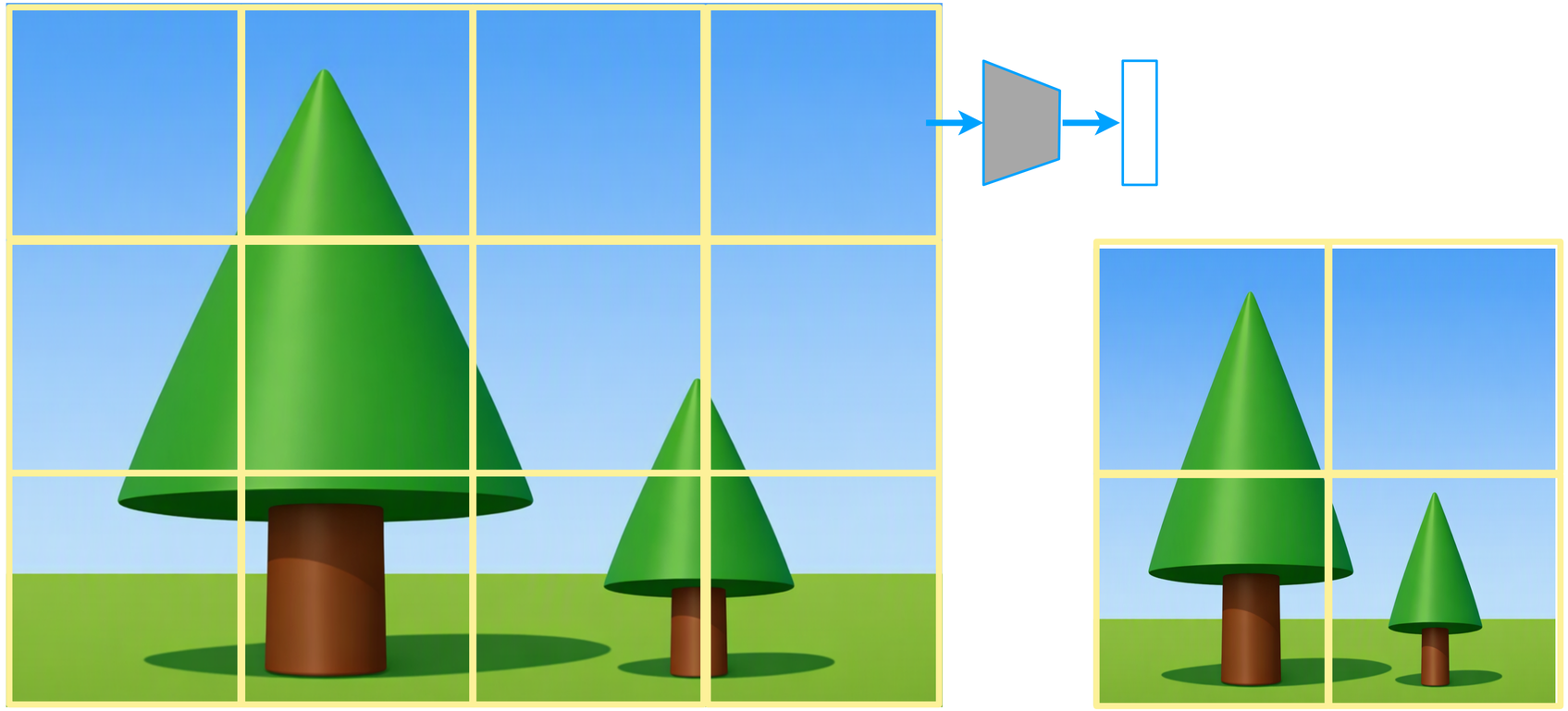
- Massively parallel (both forward and backward pass)
  - each token position is completely independent outside of attention — need only infrequent sync w/ other positions
  - sparse and linear attention variants, even more so
- Matches well w/ current architectures (clusters of GPU machines): very heavy on tensor ops (einsums), less extreme on communication latency and bandwidth
- Parallelism lets us train huge models on huge datasets
  - store a lot of information in 100B weights
  - gives access to long tail (rare but useful info)
  - lets us average out noise ($1/\sqrt{n}$ rate)

What else?

- We saw the complete Llama 3 architecture (2024)
- More recent architectures add a couple more ideas
- E.g., Qwen 3.6 (March 2026):
  - ***mixture of experts*** (MoE): activate only part of model (~10% of weights) per token (***router*** learns what part)
  - allows model to store more information (more weights) without increasing computation
  - ***linear attention***: simplified version of key→value store using only matrix multiplication, time $O(T)$ for T tokens
  - tradeoff is lossy memory, particularly for distant past — can replace 75% of attention layers, remaining 25% "real" attention can still do fine-grained search of past

Encoding images

for video, 3d grid of patches and 3d positional encoding



- Divide image into patches, send each patch through MLP to get an embedding vector
- Add positional encoding (coordinates i,j), concatenate rows of encodings, and wrap in <image> </image>
- Also rescale to a fixed size thumbnail and do the same

Learning objectives—transformers

- What is attention? Self-attention? Masking?
- What are multi-head attention and GQA?
- What is a positional encoding? How does RoPE work?
- What are all of the above good for / why do we need them?
- What are the components of a transformer model, using Llama 3 as an example?
- Why do people use variants like encoder-decoder, MoE, and linear attention?
- Why are transformers so broadly popular?